

Problem Solving & Programming - Class Notes

Table of Contents

- [Introduction](#)
- [Operators](#)
 - [First Group](#)
 - [Highest Precedence Group](#)
 - [Second Group](#)
 - [Unary Operators](#)
 - [Third Group](#)
 - [Arithmetic Operators](#)
 - [Subgroup-1](#)
 - [Subgroup-2](#)
 - [Fourth Group](#)
 - [Shift Operator](#)
 - [Fifth Group](#)
 - [Relational Operator](#)
 - [Sixth Group](#)
 - [Equality Operator](#)
 - [Seventh Group](#)
 - [Bitwise Operator](#)
 - [Eighth Group](#)
 - [Logical Operators](#)
 - [Ninth Group](#)
 - [Assignment Operator](#)
 - [Tenth Group](#)
 - [Conditional \(Ternary\) Operator](#)
 - [Eleventh Group](#)
 - [Post Increment/Decrement](#)
- [Control Structures \(Decision Making\)](#)
- [Control Statements \(Loops\)](#)
 - [For Loop](#)
 - [Break Statement](#)
 - [Continue Statement](#)
 - [While Loop](#)
 - [Do-While Loop \(Bottom Tested Loop\)](#)
 - [Nested Loops](#)

- [Pattern Printing](#)
- [Local Variable](#)
- [Global Variables](#)
- [Array](#)
- [2-D Array](#)
- [Switch Case Statement](#)
- [Go-To Statement](#)
 - [Avoiding Go-To Statement](#)
- [Pointers](#)
 - [Pointer Arithmetic](#)
 - [Addresses](#)
 - [Pointer to Array](#)
 - [Pointer to Constant](#)
 - [Constant Pointer](#)
 - [Constant Pointer to Constant Integer](#)
 - [Array of Pointers](#)
- [Stack Smashing](#)
- [Segmentation Faults](#)
- [Memory Access Violations](#)
- [Code Bloat](#)
- [Dynamic Memory Allocation](#)
- [Functions](#)
 - [Call by Value](#)
 - [Call by Reference or Call by Address](#)
 - [Default Arguments](#)
 - [Passing Arrays into Functions](#)
 - [Inline Functions](#)
- [Structures](#)
 - [Padding](#)
 - [Memory Alignment](#)
 - [Self-Referential Structures](#)
 - [Nested Structures](#)
- [Union](#)
- [Strings](#)
 - [C-Style Strings](#)
 - [Pre-Defined String Functions](#)
 - [Gets & Puts Functions](#)
 - [Avoiding Gets & Puts Functions](#)
- [Buffer Overflows](#)

- [Recursion](#)
- [Function Returning Addresses](#)
- [Algorithm](#)
 - [Characteristics of an Algorithm](#)
 - [Algorithm Performance Analysis](#)
 - [Time Complexity of a Program \(Running Time\)](#)
 - [Time Complexity of an Algorithm \(Running Time\)](#)
 - [Asymptotic Notations](#)
 - [Types of Complexity](#)
- [Command Line Arguments](#)
- [Scope Resolution Operator](#)
- [File Handling](#)
- [Flow Chart](#)
- [Exception Handling](#)
- [Storage Classes](#)
 - [Types of Storage Classes:](#)
 - [Automatic](#)
 - [Extern](#)
 - [Static](#)
 - [Register](#)
 - [Mutable](#)

Introduction

Processor, Operating System, RAM, Hard Disk Drive, Loader, ROM, Machine Dependency

Let's get an image editing software, its files are going to be stored in the Hard Disk Drive.

Executable Files are platform dependent. Executable Files have Machine Dependency. Executable Files have a Binary format (010111001101). Executable Files are stored on Hard Disk Drive

A program runs only when its executable code is in the RAM. RAM may have several Executable Files at once, this depends upon the capacity of RAM.

Operating System is pre-stored on the Hard Disk Drive

As soon as the system starts, Operating System is loaded onto the RAM. Loader is the program that loads soft-wares on the RAM.

Boot-Strap Loader is a program stored onto the ROM. This is a self-executable program that loads the Operating System onto the RAM. This whole phenomenon is known as Booting. While Booting the Boot-Strap Loader also checks the hardware configuration and health of the devices on the computer.

Boot-Strap Loader program is stored on the ROM, and cannot be updated this is a problem to solve this, companies program the ROM's BSL to load another BSL program which is stored on the Hard Disk Drive and can be updated, these are the two versions of BSL programs.

Now this whole process is known as Booting.

For Operating System Softwares where high security and ultimate malware protection is needed, Operating System is stored on the ROM, making it unalterable and hence immune to malware attacks. This phenomenon is applied in several areas like military softwares, anti missile programs, etc.

Hard Disk Drive are mechanical devices and hence slower.

All the components of Operating System are not loaded onto the RAM this is to also leave vacant space in the RAM for other softwares, for example all the levels of a game are not loaded at once and subsequently loaded at each increment but this requires loading the following part of the game onto the RAM from the Hard Disk Drive which is a slow process, hence the loading screens we get.

Any program written in any language is converted into Binary by a Compiler software which are then fed to the processor in terms of electrical signals of high and low voltages then sent to the Processor to computer and then the returning Binary signals are interpreted and outputted.

Binary is a Machine Level Language

Platform is Operating System and Processor

Machine Level Language

In the earlier days of computers the only programming language available were machine languages. While each computer still has its own machine language, which is made of strings of 0's & 1's.

Symbolic Language

A special program called an Assembler is used to translate Symbolic Language code (Source code) into Machine Level Language (Binary)

High Level Language

Compiler is a program that converts High Level Language into Machine Level Language.

Operators

1. Unary
2. Binary
3. Ternary

Unary single operator is associated with single operand, eg. -5.

Binary single operator is associated with two operands, eg. 2-5.

Ternary single operator is associated with more than two operands, eg. $a > b ? c : d$.

Precedence

Precedence refers to priority of operations, for eg. arithmetic follows BODMAS rule.

When we see the priority between the operators which belong to two or more different groups, then it is known as Operators Precedence.

Associativity

Associativity If two operators of the same priority are present.

When we see the priority between operators which belong to the same group and same subgroup, then it is known as Associativity of Operators.

First Group

Highest Precedence Group

Highest Priority

```
() Function Call Operator  
[] Subscript Operator  
. Dot or Member Access Operator  
-> Pointer to Member Access Operator
```

Second Group

Unary Operators

```
++, -- Pre-Increment/Decrement  
- Unary Minus  
sizeof Size of Data-Type  
& Address of  
* Value at the address of
```

C++

```
int a=10;  
++a;  
cout<<a;
```

Output

```
11
```

Third Group

Arithmetic Operators

Subgroup-1

```
* Multiplication  
/ Division  
% Modulus (Tells the Remainder)
```

Subgroup-2

```
+ Addition  
- Subtraction
```

Modulus Operator only Operates on Integers

C++

```
// cout << 10.5%2;  SYNTAX ERROR
```

C++

```
cout << 13%10;
```

Output

```
3
```

C++

```
int a = 523;  
int b;  
b = a % 10;  
cout<<b;
```

Output

```
3
```

Fourth Group

Shift Operator

```
<< Left Shift  
>> Right Shift
```

Left Shift operator `<<` shifts the binary form of the number to the left (adds a 0 from least significant bit) by given number of places, i.e., multiplies the given number by 2 given number of times.

5 in binary is 101

5 << 4 -> left shift by 4 places

101 0 0 0 0 or $5 * \underline{2} * \underline{2} * \underline{2} * \underline{2}$
add places multiply

101000 in decimal is 80

Also, the product as well is 80

C++

```
cout << (5<<4);
```

Output

80

Right Shift operator >> shifts the binary form of the number to the right (truncates from least significant bit) by given number of places, i.e., divides the given number by 2 given number of times.

80 in binary is 1010000

80 >> 3 -> right shift by 3 places

1010 0 0 0 or $80 / \underline{2} / \underline{2} / \underline{2}$
remove divide

1010 in decimal is 10

Also, the quotient as well is 10

C++

```
cout << (80>>3);
```

Output

10

Fifth Group

Relational Operator

```
< Less Than  
> Greater Than  
<= Less Than or Equal To  
>= Greater Than or Equal To
```

C++

```
cout << (5>10);
```

```
0
```

Sixth Group

Equality Operator

```
== Is Equal To  
!= Is Not Equal To
```

C++

```
cout << (5==10);
```

```
0
```

Seventh Group

Bitwise Operator

```
& Bitwise AND  
| Bitwise OR
```

Bitwise OR discards carry

Bitwise OR is bitwise-binary addition of given numbers.

Bitwise AND is bitwise-binary multiplication of given numbers.

Eighth Group

Logical Operators

```
&& Logical AND  
|| Logical OR
```

In logical AND, the control halts if the initial condition proves false, deeming the entire expression false. Conversely, in logical OR, the path ends upon encountering the first true condition, rendering the entire expression true.

C++

```
cout << (5>2)&&(6>3);
```

Output

```
0
```

C++

```
cout << (5>2)|| (6>3);
```

Output

```
1
```

Ninth Group

Assignment Operator

```
= Simple Assignment  
*= Multiplication Assignment  
+= Addition Assignment  
/= Division Assignment  
%= Modulus Assignment
```

C++

```
int a = 10;  
int b = 5;  
a *= b;  
cout << a;
```

Output

50

Tenth Group

Conditional (Ternary) Operator

```
(condition)?if_cond_true:if_cond_false
```

C++

```
(10>11)?cout<<"True":cout<<"False";
```

Output

False

C++

```
int a = 10;  
int b = 20;  
int c;  
c = (a<b)?b:a;  
cout<<c;
```

Output

20

C++

```
int a = 10;  
int b = 20;  
int c = 30;  
int m;
```

```
m = (a>b)?(a>c)?a:b:c;
cout<<m;
```

Output

30

Question

Write a program for finding largest of three numbers using conditional operator.

Solution

C++

```
int a,b,c,m;
cout << "Enter a : ";
cin >> a;
cout << "Enter b : ";
cin >> b;
cout << "Enter c : ";
cin >> c;

m = (a>b)?((a>c)?a:c):((b>c)?b:c);
cout << "Largest is : " <<m<<endl;
```

Question

Write a program to swap two numbers using only two variables.

Solution

C++

```
int a = 10;
int b = 15;

a = a + b; // a = 10 + 15 = 25
b = a - b; // b = 25 - 15 = 10
a = a - b; // a = 25 - 10 = 15

cout<<a<<endl; // 15
cout<<b<<endl; // 10
```

Output

15
10

Eleventh Group

Post Increment/Decrement

++, --

C++

```
int a = 5;
int j;
j = a++;
cout<<j<<endl;
cout<<a;
```

Output

5
6

Example Code:

C++

```
int m = 0;
int n = 1;
int k;

k = m&&(++n);

cout<<"k = "<<k<<" and "<<"n = "<<n<<endl;
```

Output

k = 0 and n = 1

C++

```
int m = 1;
int n = 1;
int k;

k = m&&(++n);

cout<<"k = "<<k<<" and "<<"n = "<<n<<endl;
```

Output

k = 1 and n = 2

C++

```
int m = 1;
int n = 1;
int k;

k = m || (++n);

cout<<"k = "<<k<<" and "<<"n = "<<n<<endl;
```

Output

k = 1 and n = 1

Control Structures (Decision Making)

```
if  
else  
else if
```

C++

```
int i = 10;  
if ( i == 10 )  
cout<<i;  
else  
cout<<"20";
```

Output

```
10
```

Question

Write a program to check whether the given input is even or odd.

Solution

C++

```
int a;  
cout<<"Enter a number : ";  
cin>>a;  
  
if ((a % 2) == 0)  
cout<<a<<" is an even number."<<endl;  
else  
cout<<a<<" is an odd number."<<endl;
```

Question

Write a program to check whether the given three digit number is an armstrong number.

Solution

C++

```
int n, a, b, c, d;  
cout<<"Enter a number : ";  
cin>>n;
```

```

a = n % 10;
b = ((n % 100) - a)/10;
c = n/100;

d = ((a*a*a) + (b*b*b) + (c*c*c));

if (d == n)
cout<<n<<" is an armstrong number.";
else
cout<<n<<" is not an armstrong number.";

```

Question

Write a program to check whether a given number is an armstrong number.

Solution

C++

```

int n = 1;
int sum = 0;
int temp = 0;
int temp_n = n;

while (n != 0){
temp = n % 10;
sum += temp*temp*temp;
n /= 10;
}

if (sum == temp_n)
cout<<"Given number is armstrong";
else
cout<<"Given number is not armstrong";

```

Question

Write a program to check whether the given input is a number, capital letter, small letter, or special character.

Solution

C++

```

char ch;
cout<<"Enter Input : ";
cin>>ch;

if ((ch>=48)&&(ch<=57))

```

```
cout<<"Number"<<endl;
else if ((ch>=65)&&(ch<=90))
cout<<"Capital Letter"<<endl;
else if ((ch>=97)&&(ch<=122))
cout<<"Small Letter"<<endl;
else
cout<<"Special Symbols"<<endl;
```

Control Statements (Loops)

When we want to execute a single statement or a group of statements repeatedly until some specified conditions (or for a fixed no. of times) then we can use control statements.

For Loop

syntax

C++

```
for(initialization; condition; increment/decrement) {  
    action  
}
```

Question

Write a program to calculate average of ten numbers given by the user.

Solution

C++

```
int a;  
double b=0;  
  
for (int i=1; i<=10; i++){  
    cout<<"Enter Number "<<i<<" : ";  
    cin>>a;  
    b = a + b;  
}  
  
cout<<"The average is : "<<b/10;
```

Question

Write a program to print table for one to ten numbers.

Solution

C++

```
for (int i=1; i<=10; i++){  
    cout<<"Table of "<<i<<endl<<endl;  
    for (int j=1; j<=10; j++){  
        cout<<i<<" x "<<j<<" = "<<i*j<<endl;
```

```
}  
cout<<endl;  
}
```

If several initialisation-conditions-in/decrements are given to a for loop, the right-most initialisation-condition-in/decrement will be considered by the compiler.

C++

```
for (int i=1, j=2; i<10, j<6; i++, j++)  
cout<<"Hello "; // Print 4 times for j = 2 to j = 5
```

Output

```
Hello Hello Hello Hello
```

Question

Write a program to calculate factorial of given number.

Solution

C++

```
int a;  
int b = 1;  
cout<<"Enter a number : ";  
cin>>a;  
  
for (int i = a; i > 1; i--)  
b = b*i;  
  
cout<<"Factorial of "<<a<<" is : "<<b<<endl;
```

Question

Write a program to check whether a given input is prime or not.

Solution

C++

```
int a;  
int b = 1;  
  
cout<<"Enter a number : ";  
cin>>a;
```

```

for (int i = 2; i < a; i++)
{
    if (a%i == 0){
        b = 0;
        break;
    }
}

if (b)
    cout<<a<<" is a prime number."<<endl;
else
    cout<<a<<" is a composite number."<<endl;

```

Break Statement

```
break
```

When break executes it terminates the inner-most loop of a program, break statement can only be used within control statements or in switch case statement.

Continue Statement

```
continue;
```

Continue is a statement which can only be used within the control statements(loops). When continue statement executes in a loop the control is jumped to the starting of the loop, the remaining statements are not executed.

In a for loop control is jumped to the increment/decrement statement, but in the case of while loop it jumps to the condition part.

Continue statement can only be used in 'for', 'while' and 'do-while' loops.

If the scope { } of a control statement or structure is not defined, by default the scope extends to the following line only.

C++

```

for (int i = 0; i < 3; i++)
    cout<<"Hello"<<endl;
    cout<<"World"<<endl;

```

Output

Hello
Hello
Hello
World

While Loop

syntax

C++

```
initialization;

while(condition){
    body code
    increment/decrement
}
```

C++

```
int i = 1;
while (i<=10){
    cout<<i<<endl;
    i++;
}
```

Question

Write a program to calculate the average of the age of 'n' students.

Solution

C++

```
int input_age, no_students;
int i=1;
int avg = 0;

cout<<"Enter the number of students : ";
cin>>no_students;

while(i<=no_students){
    cout<<"Enter the age of student "<<i<<" : ";
    cin>>input_age;

    if(input_age <= 0){
        cout<<"You have entered incorrect age."<<endl<<"Please try again."<<endl;
        continue;
    }
    avg = avg + input_age;
    i++;
}
```

```
}  
cout<<endl<<"Average age is : "<<avg/no_students<<endl<<endl;
```

Do-While Loop (Bottom Tested Loop)

syntax

C++

```
initialization;  
do{  
body code;  
increment/decrement;  
} while (condition);
```

C++

```
int i = 1;  
do{  
cout<<"hello";  
i++;  
} while (i<=10);
```

Output

hellohellohellohellohellohellohellohellohello

A do-while loop will always run at least once.

Question

A country, South Africa, has population 175600 and it's growth rate is 2.75% per year, while another country, Sri Lanka, has population 235900 and it's growth rate is 2.25% per year. After how many years the population of South Africa will cross the population of Sri Lanka.

Solution

C++

```
int pop_sa, pop_sl, year;  
float gr_sa, gr_sl;  
pop_sa = 175600;  
pop_sl = 235900;  
gr_sa = 2.75;  
gr_sl = 2.25;
```

```
year = 0;

while(pop_sa<pop_sl){
year++;
pop_sa = pop_sa + (pop_sa*gr_sa/100);
pop_sl = pop_sl + (pop_sl*gr_sl/100);
}

cout<<"Answer is : "<<year<<endl;
```

Output

```
Answer is : 61
```

Question

How many numbers btw. 1 to 1000 are exactly divisible by 2,5,7.

Solution

C++

```
int end, count;
end = 1000;
count = 0;

while(end>0){
if ((end%2==0)&&(end%5==0)&&(end%7==0))
count++;
end--;
}

cout<<"Answer is : "<<count<<endl;
```

Output

```
Answer is : 15
```

Nested Loops

C++

```
for (int i=1; i<=10; i++)
{
    for (int j=1; j<=10; j++)
    {

    }
}
```

Pattern Printing

C++

```
int n;
cout<<"Enter any number : ";
cin>>n;
for (int i=1; i<=n; i++){
for (int j=1; j<=i; j++){
cout<<i;
}
cout<<endl;
}
```

Output

```
1
22
333
4444
55555
```

C++

```
int n;
cout<<"Enter any number : ";
cin>>n;
for (int i=n; i>=1; i--){
for (int j=1; j<=i; j++){
cout<<i;
}
}
```

```
cout<<endl;
}
```

Output

```
55555
4444
333
22
1
```

C++

```
int n;
cout<<"Enter any number : ";
cin>>n;
for (int i=n; i>=1; i--){
for (int j=1; j<=i; j++){
cout<<"*";
}
cout<<endl;
}
```

Output

```
*****
****
***
**
*
```

C++

```
int a =1;
int n;
cout<<"Enter any number : ";
cin>>n;
for (int i=1; i<=n; i++){
for (int j=1; j<=i; j++){
cout<<a<<" ";
a++;
}
cout<<endl;
}
```

Output

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

C++

```
int n;
cout<<"Enter any number : ";
cin>>n;
for(int i = 1; i <= n; i++){
    for(int j = 1; j <= n - i; j++){
        cout<<" ";
    }
    for(int k = 1; k <= i; k++){
        cout<<i;
    }
    cout<<endl;
}
```

Output

```
1
22
333
4444
55555
```

C++

```
for(int i = 1; i <= 5; i++){
    int a = 1;
    for(int j = 1; j <= 5 - i; j++){
        cout<<" ";
    }
    for(int k = 1; k <= i; k++){
        cout<<a;
    }
    cout<<endl;
}
```

Output

```
1
12
```

```
123
1234
12345
```

C++

```
for (int i=1; i<=5; i++){
char ch = 65;
for (int j=1; j<=i; j++){
cout<<ch;
ch++;
}
cout<<endl;
}
```

Output

```
A
AB
ABC
ABCD
ABCDE
```

C++

```
for (int i=1; i<=5; i++){
    for(int j = 1; j <= 5 - i; j++){
        cout<<" ";
    }
    for (int j=1; j<=(2*i - 1); j++){
        cout<<"*";
    }
cout<<endl;
}
```

Output

```
    *
   ***
  *****
 *****
*****
```

C++

```

for (int i=5; i>=1; i--){
    for(int j = 1; j <= 5 - i; j++){
        cout<<" ";
    }
    for (int j=1; j<=(2*i - 1); j++){
        cout<<"*";
    }
    cout<<endl;
}

```

Output

```

*****
*****
****
***
**
*

```

C++

```

for (int i=5; i>=1; i--){
    char ch = 65;
    for(int j = 1; j <= 5 - i; j++){
        cout<<" ";
    }
    for (int j=1; j<=i; j++){
        cout<<ch<<" ";
        ch++;
    }
    cout<<endl;
}

```

Output

```

A B C D E
 A B C D
  A B C
   A B
    A

```

C++

```

for (int i=1; i<=5; i++){
    char ch = 65;
    for(int j = 1; j <= 5 - i; j++){
        cout<<" ";
    }
}

```

```

    }
    for (int j=1; j<=i; j++){
        cout<<ch<<" ";
        ch++;
    }
    cout<<endl;
}

```

Output

```

    A
  A B
A B C
A B C D
A B C D E

```

Question

Write a program to find out how many numbers from 2 to 'n' are prime numbers.

Solution

C++

```

int n, count_p = 0, is_p;
cout<<"Enter a number : ";
cin>>n;

for (int i = 2; i <= n; i++){
    is_p = 1;
    for (int j = 2; j < i; j++){
        if (i%j == 0){
            is_p = 0;
            break;
        }
    }
    if (is_p)
        count_p++;
}

cout<<"No. of prime numbers between 2 & "<<n<<" is : "<<count_p<<endl;

```

Question

Write a program to find out how many numbers from 2 to 'n' are armstrong numbers.

Solution

C++

```

int n, n_alt, is_a {0}, count_a {0}, t5n, t4n, t3n, t2n, t1n, sum;

cout<<"Enter a number : ";
cin>>n;

for (int i = 2; i<=n; i++){
n_alt = i;
t1n = n_alt%10;
n_alt = n_alt/10;
t2n = n_alt%10;
n_alt = n_alt/10;
t3n = n_alt%10;
n_alt = n_alt/10;
t4n = n_alt%10;
n_alt = n_alt/10;
t5n = n_alt%10;

sum = ((t1n*t1n*t1n) + (t2n*t2n*t2n) + (t3n*t3n*t3n) + (t4n*t4n*t4n) +
(t5n*t5n*t5n));

if (sum == i)
count_a++;
}
cout<<"No. of armstrong numbers btw. 2 and "<<n<<" is : "<<count_a<<endl;

```

Local Variable

- **Definition:** Local variables are declared within a specific block or function scope and can only be accessed within that scope.
- **Scope:** Local variables are accessible only within the block of code where they are declared or within the function in which they are defined.
- **Lifetime:** Their lifetime begins when the function or block in which they are declared is entered and ends when that function or block exits.
- **Storage:** Local variables are typically stored on the stack, making them fast to allocate and deallocate.
- **Example:**

C++

```
void function() {  
    int local = 10; // LOCAL VARIABLE  
    // LOCAL VARIABLE IS ACCESSIBLE ONLY WITHIN THIS FUNCTION  
}
```

Global Variables

- **Definition:** Global variables are declared outside of all functions and blocks and can be accessed from anywhere within the program.
- **Scope:** Global variables have program-wide scope, meaning they can be accessed from any function or block in the program.
- **Lifetime:** Their lifetime extends throughout the entire execution of the program. They are initialised once when the program starts and remain in memory until the program terminates.
- **Storage:** Global variables are typically stored in a special section of memory known as the data segment.
- **Example:**

| C++

```
int global = 20; // GLOBAL VARIABLE

void function() {
// GLOBAL VARIABLE IS ACCESSIBLE HERE
}
```

Array

Array is a collection of elements of similar data-types, where all the elements of array share a common name.

syntax

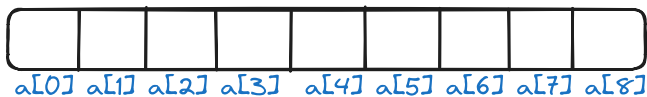
C++

```
datatype arrayname[size];
```

C++

```
int a[5];
```

```
double a[9] ;
```



1. The size of array should be specified at compile time.

Compile time is a stage in the software development process where source code written by a programmer is translated into machine code or some intermediate form that can be executed by a computer.

2. Array elements are stored in contiguous memory locations.
3. All the elements of array must be similar data-type.
4. Array indexing starts from 0.
5. Size of array is a constant unsigned integer number.
6. Name of array represents the memory location of the first element of the array.

C++

```
int a[5]; // ALL ELEMENTS OF ARRAY CAN BE ACCESSED USING SUBSCRIPT  
OPERATOR [] AND SUBSEQUENT INDEXES  
cin>>a[0];  
cin>>a[1];  
cin>>a[2];  
cin>>a[3];  
cin>>a[4];  
cout<<a[0];  
cout<<a[1];  
cout<<a[2];
```

```
cout<<a[3];  
cout<<a[4];
```

C++

```
int a[3] = {10,80,50};  
cout<<"a[0] : "<<a[0]<<endl;  
cout<<"sizeof(a[0]) : "<<sizeof(a[0])<<endl;  
cout<<"a : "<<a<<endl;  
cout<<"&a[0] : "<<&a[0]<<endl;  
cout<<"&a[1] : "<<&a[1]<<endl; // EXPECTED 4 MORE THAN &a[0] BCZ. INT  
DATA-TYPE  
cout<<"&a[2] : "<<&a[2]<<endl; // EXPECTED 4 MORE THAN &a[1] BCZ. INT  
DATA-TYPE  
cout<<"&a[2] + 1 : "<<(&a[2] + 1)<<endl; // EXPECTED 4 MORE THAN &a[2]  
BCZ. INT DATA-TYPE
```

Output

```
a[0] : 10  
sizeof(a[0]) : 4  
a : 0x7fffffff71c  
&a[0] : 0x7fffffff71c  
&a[1] : 0x7fffffff720  
&a[2] : 0x7fffffff724  
&a[2] + 1 : 0x7fffffff728
```

Question

Write a program to take 10 inputs in an array and find out maximum value in the given inputs.

Solution

C++

```
int a[10];  
int b = 0;  
  
for (int i = 0; i<10; i++){  
cout<<"Enter Number "<<i+1<<" out of 10 : ";  
cin>>a[i];  
  
if ((a[i]) > b){  
b = a[i];  
}  
}
```

```
}  
cout<<"Largest : "<<b<<endl;
```

Output

```
Enter Number 1 : 5  
Enter Number 2 : 6  
Enter Number 3 : 7  
Enter Number 4 : 98  
Enter Number 5 : 45  
Enter Number 6 : 458  
Enter Number 7 : 475  
Enter Number 8 : 55  
Enter Number 9 : 445  
Enter Number 10 : 55  
Largest : 475
```

Question

Write a program to find out top three elements of an array.

Solution

C++

```
int n, top1 = 0, top2 = 0, top3 = 0;  
cout<<"How many numbers do you want to enter : ";  
cin>>n;  
  
const int size = n;  
int array[size];  
  
for (int i = 0; i < size; i++){  
    cout<<"Enter number "<<i+1<<" : ";  
    cin>>array[i];  
}  
  
for (int i = 0; i < size; i++){  
    if (top1 <= array[i])  
        top1 = array[i];  
}  
  
for (int i = 0; i < size; i++){  
    if ((top2 <= array[i]) && (array[i] < top1))  
        top2 = array[i];  
}  
  
for (int i = 0; i < size; i++){  
    if ((top3 <= array[i]) && (array[i] < top2))
```

```

top3 = array[i];
}

cout<<"First highest number in the array is : "<<top1<<endl;
cout<<"Second highest number in the array is : "<<top2<<endl;
cout<<"Third highest number in the array is : "<<top3<<endl;

```

Output

```

How many numbers do you want to enter : 9
Enter number 1 : 98
Enter number 2 : 87
Enter number 3 : 54
Enter number 4 : 564
Enter number 5 : 5984
Enter number 6 : 49
Enter number 7 : 5
Enter number 8 : 56
Enter number 9 : 49
First highest number in the array is : 5984
Second highest number in the array is : 564
Third highest number in the array is : 98

```

Question

Write a program for binary to decimal conversion.

Solution

C++

```

char bin[256];
cout<<"Enter the binary number you want to convert to decimal : ";
cin>>bin;

int dec {0}, len;

for (len = 0; bin[len] != 0 /*'\0' NULL TERMINATION*/; len++) // FINDING
LENGTH OF BIN-STRING
continue;

for (int i = 0; i < len; i++){
int t = bin[len-1 - i] - '0'; // FLIP THE BINARY BECAUSE MSB IS AT THE
LOWEST INDEX AND THE LSB IS AT THE HIGHEST
dec += (t<<i);
}

cout<<"Decimal is : "<<dec<<endl;

```

Question

Write a program for decimal to binary conversion.

Solution

C++

```
int dec;
cout<<"Enter the number : ";
cin>>dec;

int bin[32]; // MAX RANGE FOR UNSIGNED INT IN C++

int i = 0;

// STORING LSB TO MSB
while(dec){
    bin[i++] = dec % 2;
    dec /= 2;
}

// PRINTING MSB TO LSB I.E. REVERSE ORDER
cout<<"Binary : ";
for(int k = i-1; k >= 0; k--){
    cout<<bin[k];
}
cout<<endl;
```

Question

Write a program to find frequency of a number in array.

Solution

C++

```
int n, count = 0;
cout<<"Enter the number of values you wish to enter : ";
cin>>n;
const int size = n;
int array[size];

for(int i = 0; i < size; i++){
    cout<<"Enter value "<<i+1<<" : ";
    cin>>array[i];
}

cout<<"Enter the value who's frequency you wish to find : ";
cin>>n;
```

```

for(int i = 0; i < size; i++){
    if(array[i] == n)
        count++;
}

cout<<"Frequency of "<<n<<" in array is : "<<count<<endl;

```

Question

Write a program to remove duplicate elements in an array.

Solution

C++

```

int n;
cout<<"Enter the number of values you wish to enter : ";
cin>>n;
const int size = n;
int array[size];

for(int i = 0; i < size; i++){
    cout<<"Enter value "<<i+1<<" : ";
    cin>>array[i];
}

for(int j = 0; j < size; j++){
    for(int k = 0; k < size; k++){
        if ((array[j] == array[k])&&(k!=j))
            array[k] = 0;
    }
}

for(int l = 0; l < size; l++){
    cout<<"Value "<<l+1<<" : "<<array[l]<<endl;
}

```

Question

Write a program to search element in array.

Solution

C++

```

int n, i = 0;
bool found;
cout<<"Enter the number of values you wish to enter : ";
cin>>n;

```

```

const int size = n;
int array[size];

for(i = 0; i < size; i++){
    cout<<"Enter value "<<i+1<<" : ";
    cin>>array[i];
}

cout<<"Enter the value you wish to find : ";
cin>>n;

for(i = 0; i < size; i++){
    if (array[i] == n){
        found = 1;
        break;
    }
}

if (found)
    cout<<"Element found at position : "<<i+1<<endl;
else
    cout<<"Element not in array.";

```

Question

Write a program to sort array elements.

Solution

C++

```

// This is not bubble sort

int n;
cout<<"How many values do you want to enter : ";
cin>>n;
const int size = n;
int array[size];

for (int i = 0; i < size; i++){
    cout<<"Enter Value "<<i+1<<" : ";
    cin>>array[i];
}
int is_sorted = 0;
do{
    // SORTING
    for (int i = 0; i < size - 1; i++){
        if (array[i] > array[i+1]){ // SWAP array[i] AND array[i+1]
            int temp = array[i];
            array[i] = array[i+1];

```

```
array[i+1] = temp;
}
}
// CHECKING SORT
for (int i = 0; i < size - 1; i++){
if (array[i] > array[i+1]){
is_sorted = 0;
break;
}
else{
is_sorted = 1;
}
}
} while (is_sorted == 0);

cout<<"Sorted Array"<<endl;
for (int i = 0; i < size; i++){
cout<<array[i]<<" ";
}
}
```

2-D Array

The name of a 2-D Array is a double pointer, with the row wise arrays being the addresses stored at those locations.

syntax

C++

```
datatype arrayname[row][column];
```

C++

```
int a[2][3];

for(int i = 0; i < 2; i++){
for(int j = 0; j < 3; j++){
cout<<"Enter element at location "<<i+1<<" "<<j+1<<" : ";
cin>>a[i][j];
}
}

cout<<endl<<"Entered Array : "<<endl;
for(int i = 0; i < 2; i++){
for(int j = 0; j < 3; j++){
cout<<a[i][j]<<"  ";
}
cout<<endl;
}
```

Output

```
Enter element at location 1 1 : 7
Enter element at location 1 2 : 9
Enter element at location 1 3 : 8
Enter element at location 2 1 : 5
Enter element at location 2 2 : 6
Enter element at location 2 3 : 4

Entered Array :
7  9  8
5  6  4
```

There would be no error if programmer doesn't specify the number of rows but has specified the number of columns in the initialiser list for a two-dimensional array.

Question

Write a program to add two matrices of any order.

Solution

C++

```
int r, c;
cout<<"Enter number of rows in matrix : ";
cin>>r;
cout<<"Enter number of columns in matrix : ";
cin>>c;
const int row = r;
const int column = c;
int mat_a[row][column];
int mat_b[row][column];
int mat_c[row][column];

cout<<endl<<"Matrix A"<<endl;
for(int i = 0; i < row; i++){
    for(int j = 0; j < column; j++){
        cout<<"Enter element "<<i+1<<" "<<j+1<<" : ";
        cin>>mat_a[i][j];
    }
}

cout<<endl<<"Matrix B"<<endl;
for(int i = 0; i < row; i++){
    for(int j = 0; j < column; j++){
        cout<<"Enter element "<<i+1<<" "<<j+1<<" : ";
        cin>>mat_b[i][j];
    }
}

for(int i = 0; i < row; i++){
    for(int j = 0; j < column; j++){
        mat_c[i][j] = mat_a[i][j] + mat_b[i][j];
    }
}

cout<<endl<<"Matrix A"<<endl;
for(int i = 0; i < row; i++){
    for(int j = 0; j < column; j++){
        cout<<mat_a[i][j]<<" ";
    }
    cout<<endl;
}

cout<<endl<<"Matrix B"<<endl;
```

```

for(int i = 0; i < row; i++){
for(int j = 0; j < column; j++){
cout<<mat_b[i][j]<<" ";
}
cout<<endl;
}

cout<<endl<<"Matrix A + B"<<endl;
for(int i = 0; i < row; i++){
for(int j = 0; j < column; j++){
cout<<mat_c[i][j]<<" ";
}
cout<<endl;
}

```

Question

Write a program to multiply two matrices of any order.

Solution

C++

```

int main(){

int r1, c1, r2, c2;
prog:
cout<<"Enter number of rows in matrix `A` : ";
cin>>r1;
cout<<"Enter number of columns in matrix `A` : ";
cin>>c1;
cout<<"Enter number of rows in matrix `B` : ";
cin>>r2;
cout<<"Enter number of columns in matrix `B` : ";
cin>>c2;

if(c1 != r2){
cout<<"These matrices cannot be multiplied, try again."
<<endl<<endl<<endl;
goto prog;
}

const int row_a = r1;
const int col_a_row_b = c1;
const int col_b = c2;
int mat_a[row_a][col_a_row_b];
int mat_b[col_a_row_b][col_b];
int mat_c[row_a][col_b];

```

```

cout<<endl<<"Matrix A"<<endl;
for(int i = 0; i < row_a; i++){
for(int j = 0; j < col_a_row_b; j++){
cout<<"Enter element "<<i+1<<" "<<j+1<<" : ";
cin>>mat_a[i][j];
}
}

cout<<endl<<"Matrix B"<<endl;
for(int i = 0; i < col_a_row_b; i++){
for(int j = 0; j < col_b; j++){
cout<<"Enter element "<<i+1<<" "<<j+1<<" : ";
cin>>mat_b[i][j];
}
}

cout<<endl<<"Matrix A"<<endl;
for(int i = 0; i < row_a; i++){
for(int j = 0; j < col_a_row_b; j++){
cout<<mat_a[i][j]<<" ";
}
cout<<endl;
}

cout<<endl<<"Matrix B"<<endl;
for(int i = 0; i < col_a_row_b; i++){
for(int j = 0; j < col_b; j++){
cout<<mat_b[i][j]<<" ";
}
cout<<endl;
}

int itr_row_a[col_a_row_b];
int itr_col_b[col_a_row_b];
int row_a_reached = 0;

row_a_inc:
if (row_a_reached == row_a){
cout<<endl<<"Matrix AxB"<<endl;
for(int i = 0; i < row_a; i++){
for(int j = 0; j < col_b; j++){
cout<<mat_c[i][j]<<" ";
}
cout<<endl;
}
return 0;
}

int col_b_reached = 0;
for(int count_col_a = 0; count_col_a < col_a_row_b; count_col_a ++){

```

```

itr_row_a[count_col_a] = mat_a[row_a_reached][count_col_a];
}

// FOR EACH ROW OF A COLUMN OF B MUST REACH MAXIMUM

col_b_inc:
for(int count_row_b = 0; count_row_b < col_a_row_b; count_row_b ++){
itr_col_b[count_row_b] = mat_b[count_row_b][col_b_reached];
}

int sum = 0;
for (int itr_row_a_col_b_ele = 0; itr_row_a_col_b_ele < col_a_row_b;
itr_row_a_col_b_ele ++){
sum += (itr_row_a[itr_row_a_col_b_ele] * itr_col_b[itr_row_a_col_b_ele]);
}

mat_c[row_a_reached][col_b_reached] = sum;

col_b_reached ++;

if (col_b_reached < col_b){
goto col_b_inc;
}
else if (col_b_reached = col_b){
row_a_reached ++;
goto row_a_inc;
}
}
}

```

Switch Case Statement

syntax

C++

```
switch (choice){  
case case1:  
body code 1  
break;  
case case2:  
body code 2  
break;  
default:  
body code  
break;  
}
```

Case doesn't input floating point numbers or logical operators/statements it only considers integer numbers and characters.

C++

```
int i;  
cout<<"Enter choice : ";  
cin>>i;  
  
switch(i){  
case 1:  
cout<<"1";  
break;  
case 2:  
cout<<"2";  
break;  
default:  
cout<<"default";  
break;  
}
```

C++

```
char i;  
cout<<"Enter choice : ";  
cin>>i;
```

```
switch(i){
case 'a':
cout<<"1";
break;
case 'b':
cout<<"2";
break;
default:
cout<<"default";
break;
}
```

Question

Write a program to show month name with respect to the month number, using switch-case statement.

Solution

C++

```
int n;
cout<<"Enter month number : ";
cin>>n;

switch(n){
case 1:
cout<<"January"<<endl;
break;
case 2:
cout<<"February"<<endl;
break;
case 3:
cout<<"March"<<endl;
break;
case 4:
cout<<"April"<<endl;
break;
case 5:
cout<<"May"<<endl;
break;
case 6:
cout<<"June"<<endl;
break;
case 7:
cout<<"July"<<endl;
break;
case 8:
cout<<"August"<<endl;
break;
```

```

case 9:
cout<<"September"<<endl;
break;
case 10:
cout<<"October"<<endl;
break;
case 11:
cout<<"November"<<endl;
break;
case 12:
cout<<"December"<<endl;
break;
}

```

Question

Write a program to check whether the given input is +ve, -ve or zero.

Solution

C++

```

int n;
cout<<"Enter a number : ";
cin>>n;

switch (n==0){
    case 1:
        cout<<"Zero"<<endl;
        break;
    case 0:
        switch (n>0){
            case 1:
                cout<<"Positive"<<endl;
                break;
            case 0:
                cout<<"Negative"<<endl;
                break;
        }
    }
}

```

Question

Write a program to input marks of a student in 5 subjects out of 100, and calculate his %age and grade him as follows.

75+ -> A

60-74 -> B

45-59 -> C

<59 -> D

Solution

C++

```
int n, sum, per;
sum = 0;

for(int i = 1; i<=5; i++){
    cout<<"Enter marks in subject "<<i<<" : ";
    cin>>n;
    sum = sum + n;
}

per = sum/5;

if (per >= 75)
    cout<<"A Grade";
else if ((per >= 60)&&(per <= 75))
    cout<<"B Grade";
else if ((per >= 45)&&(per <= 59))
    cout<<"C Grade";
else
    cout<<"D Grade";
```

Question

Write a program to print Pascal's Triangle to given input number of lines.

Solution

C++

```
int n;
cout<<"Enter a number : ";
cin>>n;

for (int i = 0; i < n; i++){
    int num = 1;

    for (int j = 0; j < n-i-1; j++){
        cout<<" ";
    }

    for (int k = 0; k <= i; k++){
        cout<<num<<" ";
        num = (num*(i-k))/(k+1);
    }
    cout<<endl;
}
```

Output

Enter a number : 5

1

1 1

1 2 1

1 3 3 1

1 4 6 4 1

Go-To Statement

C++

```
cout<<"first";
goto next;
cout<<"second";
next:
cout<<"last";
```

Avoiding Go-To Statement

In modern programming, the use of the `goto` statement is generally discouraged due to its potential to make code less readable and harder to maintain. The `goto` statement allows jumping to a labelled statement elsewhere in the code, which can lead to issues if not used carefully.

One common problem associated with the `goto` statement is the possibility of jumping over variable initialisation. Variables must be initialised before they are used, and the use of `goto` can disrupt this order, resulting in compilation errors.

C++

```
int main() {
    // ...

    if (1) {
        goto out1; // The jump to out1 may lead to uninitialized
variables
    }

    int sum = 0; // Initialization of sum

out1:
    // ...

    return 0;
}
```

In this example, if the `goto out1;` statement is triggered, it may lead to uninitialised variables, causing compilation errors. The comment clarifies that the jump to `out1` may potentially lead to uninitialized variables, emphasising the importance of avoiding such control flow constructs for the sake of code clarity and maintainability.

To maintain code readability and structure, it's recommended to use alternative control flow constructs such as loops and conditional statements. These alternatives provide clearer program logic and help avoid the pitfalls associated with the `goto` statement.

Pointers

Pointers are those variables which can store the address of some other variables of its own type.

A pointer can store the value of another pointer (address of some variable), but it cannot store the address of another pointer. To store the address of a pointer variable a double pointer is used.

- -> Value at the address of operator, tells the value stored at the address stored in a pointer. Also known as dereference operator.

Addresses can only be added or subtracted but they cannot be multiplied, divided, etc.

In the context of the C++ programming language, the symbol *preceding a variable, as in* `*p`, is an operator used to declare or work with pointers. A pointer is a fundamental data type that holds the memory address of another data element, typically of the same type. This operator, referred to as the 'dereference operator,' allows access to the value stored at the memory location pointed to by the variable `p`. It is crucial to understand that *p is distinct from p alone; p represents the memory address, while *p represents the value located at that address.* Any actions or operations conducted using `*p` directly affect the data held at the memory location pointed to by `p`.

Furthermore, with regard to algebraic operations involving `p`, it is important to note that performing arithmetic operations directly on a pointer variable, such as `p`, may lead to unintended results and undefined behaviour. Pointer arithmetic, when used, affects the memory address held by `p`. Adding or subtracting integers from `p` can result in a new memory address location. For example, `p + 1` points to the memory location adjacent to `p`, considering the size of the data type to which `p` points. Care must be taken to ensure that such operations adhere to the rules and constraints of the C++ language to avoid memory access violations, segmentation faults, or other undesirable outcomes.

C++

```
int a = 10;
int *p;
p = &a;
cout<<"a : "<<a<<endl;
cout<<"&a : "<<&a<<endl;
cout<<"p : "<<p<<endl;
cout<<"&p : "<<&p<<endl;
cout<<"*p : "<<*p<<endl;
a = a + a;
cout<<"a (after a = a + a) : "<<a<<endl;
cout<<"*p (after a = a + a) : "<<*p<<endl;
```

```
*p = *p + *p; // SAME AS a = a + a;
cout<<"a (after *p = *p + *p) : "<<a<<endl;
cout<<"*p (after *p = *p + *p) : "<<*p<<endl;
```

Output

```
a : 10
&a : 0x7fffffff71c
p : 0x7fffffff71c
&p : 0x7fffffff720
*p : 10
a (after a = a + a) : 20
*p (after a = a + a) : 20
a (after *p = *p + *p) : 40
*p (after *p = *p + *p) : 40
```

C++

```
int a = 10;
int b = 20;
int *q;
int *p;
p = &a;
q = &b;

cout<<endl<<"BEFORE ASSIGNING q TO p"<<endl<<endl;
cout<<"*p (value at the address of p) : "<<*p<<endl;
cout<<"*q (value at the address of p) : "<<*q<<endl;
cout<<"p : "<<p<<endl;
cout<<"q : "<<q<<endl;

p = q;

cout<<endl<<"AFTER ASSIGNING q TO p"<<endl<<endl;
cout<<"*p (value at the address of p) : "<<*p<<endl;
cout<<"*q (value at the address of p) : "<<*q<<endl;
cout<<"p : "<<p<<endl;
cout<<"q : "<<q<<endl;
// p = &q; SYNTAX ERROR
// a = p; SYNTAX ERROR
```

Output

```
BEFORE ASSIGNING q TO p

*p (value at the address of p) : 10
*q (value at the address of p) : 20
```

```
p : 0x7fffffff710
q : 0x7fffffff714
```

AFTER ASSIGNING q TO p

```
*p (value at the address of p) : 20
*q (value at the address of p) : 20
p : 0x7fffffff714
q : 0x7fffffff714
```

C++

```
int a = 10;
int *q;
int **p;

q = &a;
p = &q;

cout<<"a : "<<a<<endl; // VALUE STORED IN a
cout<<"&a : "<<&a<<endl<<endl; // MEMORY ADDRESS OF a

cout<<"p : "<<p<<endl; // VALUE STORED IN DOUBLE-POINTER p, SAME AS
`cout<<&q;`
cout<<"&p : "<<&p<<endl; // MEMORY ADDRESS OF p
cout<<"*p : "<<*p<<endl; // SAME AS cout<<q;
cout<<"**p : "<<**p<<endl; // SAME AS cout<<a;
**p = **p * **p; // SAME AS a = a * a;
cout<<"**p (after **p = **p * **p) : "<<**p<<endl; // SAME AS cout<<a;
cout<<"a (after **p = **p * **p) : "<<a<<endl<<endl;

cout<<"q : "<<q<<endl; // VALUE STORED IN POINTER q, SAME AS `cout<<a;`
cout<<"&q : "<<&q<<endl; // MEMORY ADDRESS OF q
cout<<"*q : "<<*q<<endl; // VALUE AT THE ADDRESS POINTED TO BY q
q = q + 1; // CHANGING THE MEMORY ADDRESS LOCATION POINTED TO BY THE
SINGLE-POINTER `q`
cout<<"q + 1 : "<<q<<endl;
cout<<"*q (after q = q + 1) : "<<*q<<endl;
// q = q * 1; SYNTAX ERROR
*q = *q * 2; // NOT SAME AS a = a * 2; BEACUSE THE MEMORY ADDRESS LOCATION
POINTED TO BY THE SINLGE-POINTER `q` WAS CHANGED
cout<<"*q (after *q = *q * 2) : "<<*q<<endl; // NOT EQUAL TO 20
```

Output

```
a : 10
&a : 0x7fffffff714
```

```

p : 0x7fffffff718
&p : 0x7fffffff720
*p : 0x7fffffff714
**p : 10
**p (after **p = **p * **p) : 100
a (after **p = **p * **p) : 100

q : 0x7fffffff714
&q : 0x7fffffff718
*q : 100
q + 1 : 0x7fffffff718
*q (after q = q + 1) : -10472
*q (after *q = *q * 2) : 0

```

Summary

The provided C++ code demonstrates the behaviour of pointers and double-pointers when used to manipulate memory addresses and values. Here's a summary of the key points:

1. Three integer variables are declared: `a`, `*q`, and `**p`.
2. `q` is assigned the memory address of `a`, and `p` is assigned the memory address of `q`.
3. The code then prints the values and memory addresses of `a`, `p`, `q`, and their respective dereferenced values.
4. The multiplication operation is performed on `**p`, resulting in `a` being squared. Both `**p` and `a` reflect this change.
5. Pointer arithmetic is demonstrated by incrementing `q` by 1. However, this does not have the same effect as incrementing `a` by 1, as it changes the memory location `q` points to.
6. An attempt to multiply `*q` by 2 is made, but it leads to an unexpected result due to the change in the memory location that `q` points to.

The code showcases how pointers and double-pointers can be used to manipulate memory and values but also illustrates some important nuances and potential pitfalls in pointer arithmetic and memory management.

Observations

Upon running the above code, we observe the following:

1. The memory location of `q` which is allocated just after `a` is, 4-bytes after `a` which is as expected, since `a` is an integer which is of 4-bytes.
2. The memory location of `p` which is allocated just after `q` is, 8-bytes after `q` which is as expected, since `q` is an address which is of 8-bytes.
3. The dereference operator accesses (or gets) the value stored at the memory address pointed to by the pointer.

4. Any arithmetic operations (as allowed by syntax) on dereferenced pointers `*q` or doubly-dereferenced pointers `**p` also affects the variables representing the memory addresses pointed to by said pointers.
5. Any arithmetic operations (as allowed by syntax) directly on pointers changes the memory address pointed to by the said pointer, this will result in unwanted outcomes, and hence are generally not recommended practice so as to avoid memory access violations, segmentation faults, or other undesirable outcomes.
6. When performing direct arithmetic operations on the memory locations such as `q = q + 1;` we get Stack Smashing Error (for `q = q + 5;`) and Segmentation Fault (for `q = q + 2;`) errors, as predicted.

Pointer Arithmetic

In C or C++ programming, pointers play a crucial role in manipulating memory addresses. A pointer declaration is associated with a specific data type to aid in pointer arithmetic.

Pointer arithmetic involves performing arithmetic operations on pointers to navigate through memory and access or modify data stored at different locations. By associating a data type with a pointer, we provide essential information to the compiler about the size of the data being pointed to.

For instance, if we have a pointer `int* ptr`, the compiler needs to know the size of an integer (typically 4 bytes on most systems) to accurately perform pointer arithmetic. When we increment or decrement the pointer, the compiler adjusts the memory address by the appropriate number of bytes based on the data type's size.

This association between a variable and a pointer facilitates seamless navigation through memory, allowing programmers to efficiently manipulate data structures and perform various operations.

While memory addresses themselves remain consistent regardless of the data type, associating a variable with a pointer and specifying its data type enables efficient pointer arithmetic.

Addresses

Addresses are always unsigned integer numbers, they always take memory of 8 bytes (may depend from compiler to compiler).

C++

```
int a = 10;
int *p = &a; // CAN DO
// *p = &a; SYNTAX ERROR

cout<<"sizeof(a) : "<<sizeof(a)<<endl; // EXPECTED OUTPUT 4, INT DATA-TYPE
cout<<"sizeof(*p) : "<<sizeof(*p)<<endl; // EXPECTED OUTPUT 4, INT DATA-TYPE
cout<<"sizeof(p) : "<<sizeof(p)<<endl; // EXPECTED OUTPUT 8, ADDRESS DATA-TYPE
cout<<"sizeof(&a) : "<<sizeof(&a)<<endl; // EXPECTED OUTPUT 8, ADDRESS DATA-TYPE
cout<<"sizeof(&p) : "<<sizeof(&p)<<endl; // EXPECTED OUTPUT 8, ADDRESS DATA-TYPE
```

Output

```
sizeof(a) : 4
sizeof(*p) : 4
sizeof(p) : 8
sizeof(&a) : 8
sizeof(&p) : 8
```

C++

```
int a = 10;
char b = 'z';
int *q;
char *p;
q = &a;
p = &b;

cout<<"a : "<<a<<endl;
cout<<"b : "<<b<<endl;
cout<<"&a : "<<&a<<endl;
cout<<"&b : "<<&b<<endl;
cout<<"q : "<<q<<endl; // SAME AS `cout<<&a;`
cout<<"p : "<<p<<endl; // SAME AS `cout<<&b;`

cout<<"sizeof(a) : "<<sizeof(a)<<endl; // EXPECTED OUTPUT 4, INT DATA-
```

```

TYPE
cout<<"sizeof(b) : "<<sizeof(b)<<endl; // EXPECTED OUTPUT 1, CHAR DATA-
TYPE
cout<<"sizeof(&a) : "<<sizeof(&a)<<endl; // EXPECTED OUTPUT 8, ADDRESS
DATA-TYPE
cout<<"sizeof(&b) : "<<sizeof(&b)<<endl; // EXPECTED OUTPUT 8, ADDRESS
DATA-TYPE
cout<<"sizeof(p) : "<<sizeof(p)<<endl; // EXPECTED OUTPUT 8, ADDRESS
DATA-TYPE
cout<<"sizeof(q) : "<<sizeof(q)<<endl; // EXPECTED OUTPUT 8, ADDRESS
DATA-TYPE
cout<<"sizeof(&p) : "<<sizeof(&p)<<endl; // EXPECTED OUTPUT 8, ADDRESS
DATA-TYPE
cout<<"sizeof(&q) : "<<sizeof(&q)<<endl; // EXPECTED OUTPUT 8, ADDRESS
DATA-TYPE

```

Output

```

a : 10
b : z
&a : 0x7fffffff714
&b : z

q : 0x7fffffff714
p : z

sizeof(a) : 4
sizeof(b) : 1
sizeof(&a) : 8
sizeof(&b) : 8
sizeof(p) : 8
sizeof(q) : 8
sizeof(&p) : 8
sizeof(&q) : 8

```

Explanation

Here, when outputting a `char` type `b` with `cout`, the compiler interprets the address as a pointer to a C-style string (a null-terminated character array). Since `b` is a single character, the output will be the character itself ('z') instead of its memory address.

Pointer to Array

Name of an array represents the memory location allocated to the first element of the said array.

C++

```
int a[5] = {10,2,9,6,11};

cout<<"a[0] : "<<a[0]<<endl;
cout<<"*a : "<<*a<<endl; // VALUE STORED AT THE MEMORY ADDRESS REPRESENTED
BY `a` WIZ. MEMORY ADDRESS OF FIRST ELEMENT, SAME AS `cout<<a[0];`
cout<<"a : "<<a<<endl; // SAME AS `cout<<&a[0];`
cout<<"&a[0] : "<<&a[0]<<endl; // MEMORY ADDRESS OF FIRST ELEMENT
cout<<"*(a+1) : "<<*(a+1)<<endl; // VALUE STORED AT THE MEMORY ADDRESS
REPRESENTED BY `a + 1` WIZ. MEMORY LOCATION JUST ADJACENT TO `a` AS PER
ITS DATA-TYPE, WIZ. MEMORY LOCATION OF SECOND ELEMENT, SAME AS
`cout<<a[1];`
cout<<"a[1] : "<<a[1]<<endl;
cout<<"&a[1] : "<<&a[1]<<endl; // MEMORY ADDRESS OF SECOND ELEMENT
cout<<"sizeof(a) : "<<sizeof(a)<<endl; // EXPECTED OUTPUT 20, INT DATA-
TYPE ARRAY OF SIZE 5
cout<<"sizeof(*a) : "<<sizeof(*a)<<endl; // EXPECTED OUTPUT 4, INT DATA-
TYPE, SAME AS `cout<<sizeof(a[0]);`
cout<<"sizeof(a)/sizeof(*a) : "<<sizeof(a)/sizeof(*a)<<endl; // NUMBER OF
ELEMENTS IN THE ARRAY
cout<<"&a : "<<&a<<endl; // MEMORY LOCATION OF THE ARRAY, SAME AS THE
MEMORY LOCATION OF THE FIRST ELEMENT OF THE ARRAY
cout<<"a+1 : "<<a+1<<endl; // MEMORY LOCATION JUST ADJACENT TO `a` AS PER
ITS DATA-TYPE, HERE INT, EXPECTED OUTPUT `MEMORY ADDRESS OF a + 4`
cout<<"&a+1 : "<<&a+1<<endl; // WHEN USING ARITHMETIC OPERATION IN
ADDRESSOF(NAME OF ARRAY) `&a` THE OPERATIONS ARE PERFORMED AT THE ADDRESS
OF LAST ELEMENT OF THE ARRAY. THIS IS AN EXCEPTION IN C++ LANGUAGE.
```

Output

```
a[0] : 10
*a : 10
a : 0x7fffffff710
&a[0] : 0x7fffffff710
*(a+1) : 2
a[1] : 2
&a[1] : 0x7fffffff714
sizeof(a) : 20
sizeof(*a) : 4
sizeof(a)/sizeof(*a) : 5
&a : 0x7fffffff710
```

```
a+1 : 0x7fffffff714
&a+1 : 0x7fffffff724
```

Summary

The provided C++ code demonstrates various aspects of arrays and memory addresses. Here's a summary of the key points and observations from the code:

1. An integer array `a` is defined with five elements: {10, 2, 9, 6, 11}.
2. The code explores the relationship between array elements, memory addresses, and sizes.
3. Accessing array elements:
 - `a[0]` and `*a` both provide the value of the first element (10).
 - `a[1]` and `*(a+1)` provide the value of the second element (2).
4. Memory addresses:
 - `a` or `&a[0]` provides the memory address of the first element.
 - `a+1` represents the memory location adjacent to `a` (second element's address).
 - `&a+1` behaves differently. It moves to the address of the last element of the array (an exception in C++).
5. Size-related information:
 - `sizeof(a)` gives the size of the entire array in bytes (expected output: 20).
 - `sizeof(*a)` provides the size of an individual element (expected output: 4 bytes for an integer).
 - `sizeof(a)/sizeof(*a)` calculates the number of elements in the array (expected output: 5).

C++

```
int a[5] = {10,2,9,6,11};
for (int i = 0; i <=4; i++){
cout<<"(a + "<<i<<" ) : "<<(a+i)<<endl;
cout<<"*(a + "<<i<<" ) : "<<*(a+i)<<endl;  // INCREMENTING MEMORY ADDRESS
TO GET TO THE SUCCESSIVE ADJACENT MEMORY LOCATION IN EACH ITERATION, THEN
ACCESSING THE VALUE STORED AT THOSE MEMORY ADDRESSES
}
```

Output

```
(a + 0) : 0x7fffffff710
*(a + 0) : 10

(a + 1) : 0x7fffffff714
*(a + 1) : 2
```

```
(a + 2) : 0x7fffffff7d718
*(a + 2) : 9

(a + 3) : 0x7fffffff7d71c
*(a + 3) : 6

(a + 4) : 0x7fffffff7d720
*(a + 4) : 11
```

C++

```
int a[5] = {10,2,9,6,11};
int i = 0;
int *p;
p = a; // SAME AS `p = &a;` BCZ. `a` IS AN ARRAY
for (i = 0; i<=4; i++){
cout<<"p : "<<p<<endl;
cout<<"*p : "<<*p<<endl<<endl; // INCREMENTING MEMORY ADDRESS TO GET TO
THE SUCCESSIVE ADJACENT MEMORY LOCATION IN EACH ITERATION, THEN ACCESSING
THE VALUE STORED AT THOSE MEMORY ADDRESSES
p++;
}
```

Output

```
p : 0x7fffffff7d710
*p : 10

p : 0x7fffffff7d714
*p : 2

p : 0x7fffffff7d718
*p : 9

p : 0x7fffffff7d71c
*p : 6

p : 0x7fffffff7d720
*p : 11
```

Pointer to Constant

Non-Constant Pointer to Constant Data

Pointer to constant is a pointer variable whose pointing location may or may not be declared at the time of initialisation and the value stored at the address pointed to by this pointer can not be altered through the said pointer, however the pointer variable can be later made to point to another memory address, upon which it will hold the same properties.

Initialising a pointer to constant is optional.

Initialising a constant is mandatory.

C++

```
int a = 10;
const int *ptr;
ptr = &a;

cout<<"a : "<<a<<endl;
cout<<"&a : "<<&a<<endl;
cout<<"ptr : "<<ptr<<endl;
cout<<"*ptr : "<<*ptr<<endl;
// *ptr = 20; SYNTAX ERROR
```

Output

```
a : 10
&a : 0x7fffffff71c
ptr : 0x7fffffff71c
*ptr : 10
```

Constant Pointer

Constant Pointer to Non-Constant Data

Constant pointer is a pointer variable whose pointing location has to be declared at the time of initialisation and cannot be altered later on, the value at the address pointed to by this pointer however can be change through the said pointer.

C++

```
int a = 10;
int b = 20;
int * const ptr = &a;

cout<<"*ptr (before *ptr = 50) : "<<*ptr<<endl; // SAME AS `cout<<a;`
*ptr = 50;
cout<<"*ptr (after *ptr = 50) : "<<*ptr<<endl; // SAME AS `cout<<a;`
cout<<"a : "<<a<<endl;
// ptr = &b; SYNTAX ERROR
```

Output

```
*ptr (before *ptr = 50) : 10
*ptr (after *ptr = 50) : 50
a : 50
```

Constant Pointer to Constant Integer

Constant Pointer to Constant Data

Constant pointer to constant integer is a pointer variable whose pointing location has to be declared at the time of initialisation and cannot be altered later on, also the value at the address pointed to by this pointer can not be changed through said pointer.

C++

```
int a = 10;
int b = 30;

const int *const ptr = &a;

cout<<"ptr : "<<ptr<<endl;
cout<<"*ptr : "<<*ptr<<endl;

// *ptr = 20; SYNTAX ERROR
// ptr = &b; SYNTAX ERROR
```

Output

```
ptr : 0x7fffffff718
*ptr : 10
```

Array of Pointers

C++

```
int *ptr[3];

int a{10}, b{30}, c{80};
ptr[0] = &a;
ptr[1] = &b;
ptr[2] = &c;

for (int i = 0; i<=2; i++){
    cout<<"ptr["<<i<<"] : "<<ptr[i]<<endl;
    cout<<"*ptr["<<i<<"] : "<<*ptr[i]<<endl;
    cout<<"&ptr["<<i<<"] : "<<&ptr[i]<<endl;
    cout<<endl;
}
```

Output

```
ptr[0] : 0x7fffffff700
*ptr[0] : 10
&ptr[0] : 0x7fffffff710

ptr[1] : 0x7fffffff704
*ptr[1] : 30
&ptr[1] : 0x7fffffff718

ptr[2] : 0x7fffffff708
*ptr[2] : 80
&ptr[2] : 0x7fffffff720
```

C++

```
char ch = 'A';
int *p;

// p = &ch; SYNTAX ERROR

p = (int *)&ch; // TYPECASTING

cout<<"&ch : "<<&ch<<endl; // WHY NOT A MEMORY ADDRESS ?
cout<<"p : "<<p<<endl;
```

Output

```
&ch : A  
p : 0x7fffffff71f
```

Stack Smashing

Not in Syllabus

Stack smashing, also known as a stack buffer overflow or buffer overrun, is a type of software security vulnerability that occurs when a program writes more data to a stack buffer (a local variable or an array) than it can hold. The "stack" here refers to the call stack, a region of memory used for function call management.

Here's an overview of how stack smashing typically occurs:

- 1. Local Variables and Buffers:**

In C and C++ programming, functions often use local variables and buffers allocated on the stack. For example, an array might be declared as `char buffer[100]`.

- 2. Insufficient Bounds Checking:**

If a program does not properly validate or check the bounds of user input before copying it into a buffer, it can lead to a buffer overflow. This is especially common with functions like `strcpy` or `sprintf`, which do not inherently check the size of the destination buffer.

- 3. Writing Beyond the Buffer:**

When a program writes more data than the buffer can hold, it overflows into adjacent memory on the stack. This can overwrite other local variables, function return addresses, or control data.

- 4. Exploitation:**

An attacker can take advantage of a stack buffer overflow to inject malicious code into the program's memory. By carefully crafting the input, they can overwrite the return address on the stack, redirecting the program's execution flow to their injected code.

- 5. Security Implications:**

Stack smashing can lead to security vulnerabilities such as arbitrary code execution, privilege escalation, denial-of-service attacks, or information leakage.

- 6. Prevention:**

To prevent stack smashing, it's crucial to use secure coding practices, validate input sizes, and employ functions that perform bounds checking. Additionally, modern compilers and tools often include features like stack canaries and address space layout randomisation (ASLR) to mitigate the impact of buffer overflows.

It's essential for developers to be aware of these vulnerabilities and adopt secure coding practices to minimize the risk of stack smashing in their software.

Segmentation Faults

Not in Syllabus

A segmentation fault (often referred to as segfault) is a specific kind of error caused by accessing memory that "does not belong to you." It's a mechanism that protects your program from causing more serious problems (like data corruption) by stopping the execution of the program when an invalid memory access is detected.

1. Memory Segmentation:

- In computer memory, a program's address space is divided into several segments: code segment, data segment, stack segment, and heap segment. Each of these segments serves a specific purpose.

2. Access Violation:

- When a program tries to access a memory location outside the boundaries of the segments to which it has permission, a segmentation fault occurs.

3. Causes of Segmentation Faults:

- **Dereferencing NULL or Invalid Pointers:** Trying to access the value pointed to by a pointer that is NULL or points to an invalid memory location.
- **Buffer Overflow:** Writing more data into a buffer than it can hold, leading to overwriting adjacent memory.
- **Stack Overflow:** Exhausting the available stack space, often caused by infinite recursion or excessive use of stack memory.
- **Use After Free:** Accessing memory that has already been freed, leading to undefined behavior.
- **Memory Corruption:** Modifying memory in a way that disrupts the structure of the program.

4. Debugging Segmentation Faults:

- Debugging segfaults can be challenging because they often involve low-level memory issues.
- Tools like gdb (GNU Debugger) can be useful for analyzing the state of the program at the time of the crash.

5. Preventing Segmentation Faults:

- Follow good programming practices.
- Be cautious with pointers and memory allocations.
- Use tools like static analyzers and memory debuggers.
- Enable compiler warnings and treat them as errors.

Here's a simple example in C++ to illustrate a segmentation fault:

C++

```
#include <iostream>

int main() {
    int* ptr = nullptr;
    *ptr = 5; // Dereferencing a nullptr, causing a segmentation fault
    return 0;
}
```

In this example, attempting to dereference a NULL pointer leads to a segmentation fault because the program is trying to access memory it doesn't have permission to access.

Memory Access Violations

Not in Syllabus

Memory access violations occur when a program attempts to access a region of memory in a way that is not allowed. These violations can lead to program crashes, unpredictable behavior, or security vulnerabilities. Here's a breakdown of memory access violations:

1. Memory Protection:

Modern operating systems use memory protection mechanisms to isolate different parts of a program's memory. This helps prevent one part of the program from accidentally or maliciously interfering with another.

2. Common Causes:

- **Dereferencing Null Pointers:** Attempting to access or modify the data at a memory address that is currently set to null.
- **Out-of-Bounds Array Access:** Trying to read or write beyond the bounds of an array.
- **Accessing Freed Memory:** Using a pointer that points to memory that has already been deallocated (freed).
- **Writing to Read-Only Memory:** Attempting to modify data in a region of memory marked as read-only.
- **Executing Data as Code:** Trying to execute data as if it were executable code.

3. Consequences:

- **Crashes:** Memory access violations often lead to program crashes. The operating system might terminate the program to prevent further issues.
- **Undefined Behavior:** In some cases, the program might continue running with unpredictable results. This is considered undefined behavior and can be challenging to debug.

4. Debugging:

- **Memory Debugging Tools:** Developers use tools like debuggers, memory profilers, and sanitizers to identify and fix memory access violations.
- **Code Review:** Careful code review can catch potential violations before they become issues.

5. Security Implications:

- Memory access violations can be exploited by attackers to gain unauthorized access, inject malicious code, or compromise the integrity of a system.

6. Prevention:

- **Pointer Checks:** Always check pointers for null values before dereferencing them.
- **Array Bounds Checking:** Ensure that array indices are within the valid range.
- **Memory Management:** Properly allocate and deallocate memory, and avoid using

pointers that point to freed memory.

Memory access violations are critical issues that require attention during software development to ensure program stability, reliability, and security.

Code Bloat

Not in Syllabus

Code bloat refers to the phenomenon where the size of a compiled program is significantly larger than expected or necessary. It typically occurs due to excessive or inefficient use of resources, such as memory or disk space, resulting in a bloated executable file.

Several factors can contribute to code bloat:

1. **Inline Expansion:** While inline functions can improve performance by eliminating function call overhead, excessive use of inline functions can lead to code bloat. When functions are inlined, the compiler inserts the function code directly into the calling code, which can result in duplicated code segments across multiple call sites, increasing the overall size of the executable.
2. **Template Metaprogramming:** Templates in C++ allow for generic programming, but they can also lead to code bloat if not used judiciously. Template instantiation can generate multiple copies of the same code for different template arguments, leading to redundant code segments in the executable.
3. **Unused Code:** Including unnecessary or unused code in the program can contribute to code bloat. This often happens when libraries or header files are included indiscriminately, resulting in functions or classes being compiled into the executable even if they are not used.
4. **Debug Information:** Compilers typically include debug information in the executable to aid in debugging. While useful during development, this debug information can significantly increase the size of the executable. Stripping debug symbols from the final executable can help reduce code bloat.
5. **Compiler Optimization Settings:** Compiler optimization settings can influence the size of the generated code. In some cases, aggressive optimization settings may lead to code bloat as the compiler prioritizes performance over code size.
6. **Third-party Libraries:** Using third-party libraries or frameworks that are not optimized for code size can also contribute to code bloat. These libraries may include features or dependencies that are not needed for the specific application, resulting in unnecessary overhead.

To mitigate code bloat, developers should carefully review their codebase, avoid excessive use of inline functions and templates, remove unused code, optimize compiler settings for code size, and use libraries that are tailored to the requirements of the application. Additionally, tools like static analyzers and profilers can help identify and address code bloat issues in large codebases.

Dynamic Memory Allocation

malloc

malloc function allocates the memory at runtime and returns the starting address of allocated block (address is by default void type). malloc function takes only one argument i.e. no. of bytes to be allocated.

syntax

C++

```
malloc(no. of bytes you want to allocate)
```

Static Memory Allocation = Compile Time Memory Allocation

Dynamic Memory Allocation = Run Time Memory Allocation

C++

```
int a[10];
int b;
float c;
char d;
```

C++

```
int *ptr; // CREATING A POINTER VARIABLE FOR MEMORY ALLOCATION THROUGH
`malloc`
int i,n;
cout<<"How many numbers do you want to enter : ";
cin>>n;

ptr = (int *)malloc(n*sizeof(int)); // ALLOCATING MEMORY ENOUGH TO STORE
`n` VARIABLES THROUGH `malloc` AND POINTING `ptr` TO START OF SAID MEMORY
LOCATION

for (i = 0; i < n; i++){
cout<<"Enter Value "<<i+1<<" : ";
cin>>*(ptr+i); // STORING CONTIGUOUS INTEGER VALUES IN ADJACENT MEMORY
LOCATIONS STARTING FROM THE MEMORY LOCATION POINTED TO BY `ptr`
}

for (i = 0; i < n; i++){
cout<<"Value "<<i+1<<" : "<<*(ptr+i)<<endl; // ACCESSING ABOVE STORED
```

VALUES AND PRINTING THEM

```
cout<<"*(ptr + "<<i<<" ) : "<<*(ptr+i)<<endl;  
}
```

Output

```
How many numbers do you want to enter : 4  
Enter Value 1 : 2  
Enter Value 2 : 5  
Enter Value 3 : 69  
Enter Value 4 : 878  
Value 1 : 2  
*(ptr + 0) : 2  
Value 2 : 5  
*(ptr + 1) : 5  
Value 3 : 69  
*(ptr + 2) : 69  
Value 4 : 878  
*(ptr + 3) : 878
```

Summary

This code demonstrates dynamic memory allocation using the `malloc` function to store and retrieve integer values. Here's a summary of the code:

1. Declare an integer pointer variable `ptr` to be used for dynamic memory allocation.
2. Declare integer variables `i` and `n` to be used for loop control and storing the number of values to be entered.
3. Prompt the user to enter the number of values they want to input and store it in the variable `n`.
4. Allocate memory using `malloc` to store `n` integer values. The memory allocated is sufficient to store `n` integers and is pointed to by the `ptr` variable.
5. Use a `for` loop to iterate from 0 to `n` (inclusive) for entering values. Within the loop:
 - Prompt the user to enter each value, starting from value number 1, and store it in the memory location pointed to by `ptr + i`. This allows for storing contiguous integer values in adjacent memory locations.
6. Use another `for` loop to iterate from 0 to `n` (inclusive) for displaying the stored values. Within the loop:
 - Access and display the stored values using `*(ptr + i)`, representing the value at the `i`-th position.

C++

```
int *ptr;  
char ch;
```

```

do {
ptr = (int *)malloc(4); // CREATING ENOUGH SPACE TO STORE INTEGER (4
BYTES) IN EACH ITERATION, AND THEN ALLOCATING IT'S MEMORY LOCATION TO
POINTER `ptr`, THIS IS EXPECTED TO CHANGE IN EVERY ITERATION; THE LOOP
WILL KEEP ITERATING EVERY TIME USER PRESSES 'y'

cout<<"Enter value : ";
cin>>*ptr;
cout<<"Entered value is : "<<*ptr<<endl;
cout<<"Address of the entered value is : "<<ptr<<endl;

cout<<"To enter more values press 'y' : ";
cin>>ch;
} while ((ch=='y')||(ch=='Y'));

```

Output

```

Enter value : 44
Entered value is : 44
Address of the entered value is : 0x55555556aeb0
To enter more values press 'y' : y
Enter value : 58
Entered value is : 58
Address of the entered value is : 0x55555556b6f0
To enter more values press 'y' : y
Enter value : 8
Entered value is : 8
Address of the entered value is : 0x55555556b710
To enter more values press 'y' : n

```

Functions

A function is a self-contained block of code that performs a coherent task of some kind.

C++

```
int main() {  
    // THIS IS A FUNCTION NAMED `main`  
}  
  
int show () {  
    // THIS IS A FUNCTION NAMED `show`  
}  
  
int paste () {  
    // THIS IS A FUNCTION NAMED `paste`  
}
```

C++

```
int main () {  
    int test();  
    int show();  
    cout<<"Hello"<<endl;  
    show();  
    cout<<"HI"<<endl;  
    test();  
    return 0;  
}  
int show(){  
    cout<<"In show function"<<endl;  
}  
int test(){  
    cout<<"In test function"<<endl;  
    show();  
}
```

Output

```
Hello  
In show function  
HI  
In test function  
In show function
```

C++

```

int main(){
int a = 10, b = 20;
int c;

int add(int, int);
c = add(a,b);
cout<<"c : "<<c<<endl;

return 0;
}

int add(int i, int j){
cout<<"`add` Function"<<endl;
return (i+j);
}

```

Output

```

`add` Function
c : 30

```

C++

```

int main(){
int n;
int func();
n = func();
cout<<n<<endl;
return 0;
}

int func(){
return (3.14);
}

```

Output

```

3

```

C++

```

int main(){
int n;
int func();
n = func();

```

```
cout<<n<<endl;
return 0;
}

int func(){
return (3.14, 5.14);
}
```

Output

5

C++

```
int main(){
float n;
float func();
n = func();
cout<<n<<endl;
return 0;
}

float func(){
return (3.14, 5.14);
}
```

Output

5.14

Call by Value

Inputting values as `func(arg_val1, arg_val2)` in a `func(parameter_dtype1, parameter_dtype2)` kind function, is known as call-by-value. In this parameter passing method the function receives a copy of the argument's value. Changes made to this value don't affect the original argument variable's value (data outside the function).

This allows us to work with function's parameters, which are independent of the arguments passed to it, this shall be of use when we wish to manipulate quantities yet keeping the source unaltered.

C++

```
int main(){
    int a = 10, b = 20;
    int flip(int, int);
    flip(a,b);
    cout<<"a (in main function) : "<<a<<endl; // a = 10
    cout<<"b (in main function) : "<<b<<endl; // b = 20
    return 0;
}

int flip(int i, int j){
    int t;
    t = i; // ASSIGNING `i` IN `t`; i = 10, j = 20, t = 10
    i = j; // ASSIGNING `j` IN `i`; i = 20, j = 20, t = 10
    j = t; // ASSIGNING `t` IN `j`; i = 20, j = 10, t = 10
    cout<<"i (in flip function) : "<<i<<endl; // i = 20
    cout<<"j (in flip function) : "<<j<<endl; // j = 10
}
```

Output

```
i (in flip function) : 20
j (in flip function) : 10
a (in main function) : 10
b (in main function) : 20
```

Call by Reference or Call by Address

Inputting values as `func(&arg1 {arg_mem_address1}, &arg2 {arg_mem_address2})` in a `func(parameter_ptr_datatype1 *, parameter_ptr_datatype2 *)` kind function, is known as call-by-reference. In this parameter passing method, the function receives the memory address of the arguments, via pointers. Changes made to the values stored at the addresses pointed to by said pointers, affect the argument variable's original value (data outside the function), this is because both the argument and parameter share the same memory address.

This allows us to manipulate quantities via a pre-defined function, helping reduce redundancies, this shall be of use when we wish to manipulate a large number of values in a certain specific way.

C++

```
int main(){
    int a = 10, b = 20;
    cout<<endl<<"a (before reallocation) : "<<a<<endl;
    cout<<"b (before reallocation) : "<<b<<endl;
    void flip(int *, int *);
    flip(&a,&b);
    cout<<endl<<"a (after reallocation) : "<<a<<endl;
    cout<<"b (after reallocation) : "<<b<<endl;
    return 0;
}

void flip(int *i, int *j){
    cout<<endl<<"i (before reallocation) : "<<i<<endl;
    cout<<"j (before reallocation) : "<<j<<endl;
    cout<<"*i (before reallocation) : "<<*i<<endl;
    cout<<"*j (before reallocation) : "<<*j<<endl;
    int t;
    t = *i; // ASSIGNING THE VALUE POINTED TO BY `i` IN `t`
    *i = *j; // ASSIGNING THE VALUE POINTED TO BY `j` IN THE VALUE POINTED TO BY `i`
    *j = t; // ASSIGNING THE VALUE OF `t` IN THE VALUE POINTED TO BY `j`
    cout<<endl<<"i (after reallocation) : "<<i<<endl;
    cout<<"j (after reallocation) : "<<j<<endl;
    cout<<"*i (after reallocation) : "<<*i<<endl;
    cout<<"*j (after reallocation) : "<<*j<<endl;
}
```

Output

```
a (before reallocation) : 10
b (before reallocation) : 20

i (before reallocation) : 0x7fffffff720
j (before reallocation) : 0x7fffffff724
*i (before reallocation) : 10
*j (before reallocation) : 20

i (after reallocation) : 0x7fffffff720
j (after reallocation) : 0x7fffffff724
*i (after reallocation) : 20
*j (after reallocation) : 10

a (after reallocation) : 20
b (after reallocation) : 10
```

Summary

Code Summaries#^ee1bf8

Functions except `void` always return a value, regardless of assignment of return value in source code.

C++

```
int main (){
int k = 10;
int show();
cout<<"k (before `k = show();`) : "<<k<<endl; // EXPECTED VALUE 10
k = show();
cout<<"k (after `k = show();`) : "<<k<<endl; // EXPECTED GARBAGE VALUE
}

int show(){
cout<<"in show function"<<endl; // NO RETURN VALUE SPECIFIED, EXPECTED TO
RETURN SOME GARBAGE VALUE
}
```

Output

```
k (before `k = show();`) : 10
in show function
k (after `k = show();`) : 1431666752
```

Default Arguments

When a program omits an argument for a parameter with a default argument in a function call, the compiler re-writes the function call and inserts the default value of that argument.

Passing Arrays into Functions

C++

```
int main(){
int a[] = {20,10};
void show(int *);
show(a);
return 0;
}
void show(int *p){
cout<<*p;
p++;
cout<<*p;
}
```

Output

```
2010
```

C++

```
int main(){
void show(int *, int);

int n;
cout<<"Enter the number of values you wish to enter : ";
cin>>n;
const int size = n;
int array[size];

for(int i = 0; i < size; i++){
cout<<"Enter value "<<i+1<<" : ";
cin>>array[i];
}

show(array, size);
return 0;
}
```

```
void show(int *p, int size){
for(int i = 0; i < size; i++){
cout<<"Value "<<i+1<<" : "<<*p<<endl;
p++;
}
}
```

Output

```
Enter the number of values you wish to enter : 5
Enter value 1 : 10
Enter value 2 : 20
Enter value 3 : 30
Enter value 4 : 40
Enter value 5 : 50
Value 1 : 10
Value 2 : 20
Value 3 : 30
Value 4 : 40
Value 5 : 50
```

C++

```
int main(){
int a [10] {1,2,3,4,5,6,7,8,9,10};
show(a, 10);
}

void show(int arr[], int arr_size){
for (int i = 0; i < arr_size; i++)
cout<<arr[i]<<" ";
cout<<endl;
}
```

Using the sizeof operator in a function to find the size in bytes of an array parameter results in the size in bytes of a pointer, not the size in bytes of the array.

Inline Functions

Inline functions are a feature in C++ that allows the compiler to insert the complete body of a function into the calling code rather than executing a function call. When a function is declared as inline, the compiler reviews this request and if approved it replaces each call to that function with the actual code of the function body. This eliminates the overhead associated with the function call, such as pushing arguments onto the stack and jumping to the function code.

Function inlining is a request made to the compiler, not a command, i.e., it does not guarantee the inlining of a function.

Compilers can inline code for which you have not explicitly used the inline keyword.

Today's optimising compilers are so sophisticated that it's best to leave inlining decisions to the compiler

C++

```
#include <iostream>
using namespace std;

inline int add(int a, int b); // INLINE FUNCTION PROTOTYPE DECLARATION

int main() {
    int num1 = 5, num2 = 10;

    int result = add(num1, num2);

    cout<<"Result of addition : "<<result<<endl;

    return 0;
}

inline int add(int a, int b){ // INLINE FUNCTION DEFINITION
    return a + b;
}
```

Output

```
Result of addition : 15
```

Here are some key points about inline functions:

1. **Performance Optimisation:** Inline functions are primarily used for performance optimisation. By eliminating the overhead of function calls, especially for small and

frequently used functions, inline functions can improve the execution speed of a program.

2. **Small Functions:** Inline functions are most effective for small functions that are called frequently. Since the function code is inserted directly into the calling code, the overhead of function calls is reduced, resulting in faster execution.
3. **Function Inlining:** The decision to inline a function is made by the compiler, based on various factors such as the function's size, complexity, and the optimisation settings. While declaring a function as inline suggests to the compiler that it should be considered for inlining, the compiler ultimately decides whether to honour the request.
4. **Function Definition:** Inline functions are typically defined in the header file along with their declaration. This allows the compiler to see the function definition at the point of call in different translation units, enabling it to inline the function if deemed appropriate.
5. **Trade-offs:** While inline functions can improve performance by reducing function call overhead, they may also increase code size. Inlining large or complex functions can lead to code bloat, resulting in larger executable files and potentially slower performance due to increased memory usage and cache misses.
6. **Inlining Guidelines:** It's essential to use inline functions judiciously and selectively. Inline functions should be used for small, frequently called functions where the performance benefits outweigh the potential drawbacks. Additionally, profiling tools can help identify performance bottlenecks and determine which functions are suitable candidates for inlining.

Overall, inline functions provide a mechanism for performance optimisation in C++, allowing developers to balance code readability and maintainability with execution speed. By carefully choosing which functions to inline, developers can achieve optimal performance without sacrificing code clarity.

Structures

A structure is a user-defined data type that allows you to group together different variables under a single name. Each variable within the structure is known as a member, and these members can have different data types.

`syntax`

C++

```
struct name_of_structure{  
    declaration of variables  
};  
struct name_of_structure /*blank space*/ variable_representing_structure;
```

C++

```
struct name_of_structure{  
    declaration of variables  
} variable_representing_structure;
```

1. We can use multiple variables to represent a single structure.
2. Members of a structure can be accessed using the member access `.` (dot) operator on a structure variable.
3. Members of a structure can be accessed using the pointer to member access `->` operator on a structure pointer variable.
4. An array may also be employed to store multiple variables representing a structure.
5. We can define a structure inside another structure, also we can define a structure variable inside another structure.
6. Memory is allocated to a structure only when its variable is declared, if the variable is not declared memory will not be allocated to the structure and such a scenario is of no use.
7. Memory allocation in all variables of a structure is contiguous.
8. If variable representation of structure by array, same contiguous memory allocation rule followed.
9. Size of a structure is greater than or equal to sum size of all variables contained in it, this is because of memory alignment and padding.
10. We can also assign the values of one variable of a structure into another variable of the same structure using assignment operator.
11. The starting memory address of a structure and the starting address of its first element are the same.

12. A variable of a structure, unlike the name of an array, does not represent its starting address.
13. A structure variable cannot be declared inside the structure itself.
14. You can use a pointer of type `struct var_name` to point to the memory address of a variable of type `struct var_name` only.

C++

```
struct book{
char name[20];
int no_pages;
float price;
};
struct book e1;

cout<<"Enter name : ";
cin>>e1.name;
cout<<"Enter no of pages : ";
cin>>e1.no_pages;
cout<<"Enter the price : ";
cin>>e1.price;

cout<<endl<<"Name : "<<e1.name<<endl;
cout<<"No. of Pages : "<<e1.no_pages<<endl;
cout<<"Price : "<<e1.price<<endl;
```

Output

```
Enter name : Lotus
Enter no of pages : 550
Enter the price : 650.00

Name : Lotus
No. of Pages : 550
Price : 650
```

C++

```
struct book{
char name[20];
int no_pages;
float price;
};

struct book e1 = {"Lotus", 500, 650};

cout<<"sizeof(book) : "<<sizeof(book)<<endl;
```

```

cout<<"sizeof(e1) : "<<sizeof(e1)<<endl;
cout<<"sizeof(e1.name) : "<<sizeof(e1.name)<<endl;
cout<<"sizeof(e1.no_pages) : "<<sizeof(e1.no_pages)<<endl;
cout<<"sizeof(e1.price) : "<<sizeof(e1.price)<<endl;
cout<<endl;
cout<<"Size of all variables of the structure `book` : "
<<sizeof(e1.name)+sizeof(e1.no_pages)+sizeof(e1.price)<<endl;
cout<<"Size of the variable represnting the structure `book` : "
<<sizeof(e1)<<endl;

```

Output

```

sizeof(book) : 28
sizeof(e1) : 28
sizeof(e1.name) : 20
sizeof(e1.no_pages) : 4
sizeof(e1.price) : 4

Size of all variables of the structure `book` : 28
Size of the variable represnting the structure `book` : 28

```

C++

```

struct book{
char name[20];
int no_pages;
float price;
};

struct book e1 = {"Lotus", 500, 650};

// cout<<"&book : "<<&book<<endl; SYNTAX ERROR
cout<<"&e1 : "<<&e1<<endl;
cout<<"&e1.name : "<<&e1.name<<endl;
cout<<"&e1.no_pages : "<<&e1.no_pages<<endl; // EXPECTED 20 AFTER
`&e1.name`
cout<<"&e1.price : "<<&e1.price<<endl; // EXPECTED 4 AFTER `&e1.no_pages`

```

Output

```

&e1 : 0x7fffffff6a0
&e1.name : 0x7fffffff6a0
&e1.no_pages : 0x7fffffff6b4
&e1.price : 0x7fffffff6b8

```

C++

```

struct book{
char name[20];
int no_pages;
float price;
} e1, e2, e3;

e1 = {"Lotus", 500, 650};
e2 = {"Lily", 400, 550};
e3 = {"Rose", 300, 450};

cout<<"&e1 : "<<&e1<<endl;
cout<<"&e1.name : "<<&e1.name<<endl;
cout<<"&e1.no_pages : "<<&e1.no_pages<<endl; // EXPECTED 20 AFTER
`&e1.name`
cout<<"&e1.price : "<<&e1.price<<endl; // EXPECTED 4 AFTER `&e1.no_pages`
cout<<endl;
cout<<"&e2 : "<<&e2<<endl;
cout<<"&e2.name : "<<&e2.name<<endl;
cout<<"&e2.no_pages : "<<&e2.no_pages<<endl; // EXPECTED 20 AFTER
`&e2.name`
cout<<"&e2.price : "<<&e2.price<<endl; // EXPECTED 4 AFTER `&e2.no_pages`
cout<<endl;
cout<<"&e3 : "<<&e3<<endl;
cout<<"&e3.name : "<<&e3.name<<endl;
cout<<"&e3.no_pages : "<<&e3.no_pages<<endl; // EXPECTED 20 AFTER
`&e3.name`
cout<<"&e3.price : "<<&e3.price<<endl; // EXPECTED 4 AFTER `&e3.no_pages`

```

Output

```

&e1 : 0x7fffffff660
&e1.name : 0x7fffffff660
&e1.no_pages : 0x7fffffff674
&e1.price : 0x7fffffff678

&e2 : 0x7fffffff680
&e2.name : 0x7fffffff680
&e2.no_pages : 0x7fffffff694
&e2.price : 0x7fffffff698

&e3 : 0x7fffffff6a0
&e3.name : 0x7fffffff6a0
&e3.no_pages : 0x7fffffff6b4
&e3.price : 0x7fffffff6b8

```

C++

```

struct book {
int i; // 4 BYTES
float j; // 4 BYTES
} e1 = {10, 30}, e2, e3; // 8 BYTES

cout<<"e2.i (before `e1=e2`) : "<<e2.i<<endl; // GARBAGE VALUE
cout<<"e2.j (before `e1=e2`) : "<<e2.j<<endl; // GARBAGE VALUE
e2 = e1;
cout<<"e2.j (after `e1=e2`) : "<<e2.i<<endl; // 10
cout<<"e2.j (after `e1=e2`) : "<<e2.j<<endl; // 30

```

Output

```

e2.i (before `e1=e2`) : 1578734720
e2.j (before `e1=e2`) : 4.5912141
e2.j (after `e1=e2`) : 10
e2.j (after `e1=e2`) : 30

```

C++

```

struct book{
char name[10]; // TAKES 10 BYTES OF MEMORY
int no_pages; // TAKES 4 BYTES OF MEMORY
float price; // TAKES 4 BYTES OF MEMORY
} e1; // SHALL TAKE 18 BYTES OR MEMORY, 10 BYTES OF CHAR ARRAY,
4 BYTES OF INT AND 4 BYTES OF FLOAT

cout<<"sizeof(book) : "<<sizeof(book)<<endl;
cout<<"sizeof(e1) : "<<sizeof(e1)<<endl;
cout<<"sizeof(e1.name) : "<<sizeof(e1.name)<<endl;
cout<<"sizeof(e1.no_pages) : "<<sizeof(e1.no_pages)<<endl;
cout<<"sizeof(e1.price) : "<<sizeof(e1.price)<<endl;
cout<<endl;
cout<<"Size of all variables of the structure `book` : "
<<sizeof(e1.name)+sizeof(e1.no_pages)+sizeof(e1.price)<<endl;
cout<<"Size of the variable represnting the structure `book` : "
<<sizeof(e1)<<endl;

```

Output

```

sizeof(book) : 20
sizeof(e1) : 20
sizeof(e1.name) : 10
sizeof(e1.no_pages) : 4
sizeof(e1.price) : 4

```

```
Size of all variables of the structure `book` : 18  
Size of the variable represnting the structure `book` : 20
```

Padding

not in syllabus

1. Definition:

- **Padding** refers to the insertion of additional bytes between the members of a structure or at the end of a structure to ensure that each member is properly aligned in memory.

2. Purpose of Padding:

- **Alignment Requirements:** Different data types have different alignment requirements. For example, some processors may require that an `int` starts at an address that is a multiple of 4. Padding is used to satisfy these alignment requirements.
- **Efficient Memory Access:** Properly aligned data can be accessed more efficiently by the CPU. Misaligned data may require multiple memory accesses or additional processing, leading to performance penalties.

3. How Padding Works:

- When a structure is defined, the compiler determines the size and alignment requirements of each member.
- The decision to add padding is made by the compiler, and it's influenced by factors like the size and data types of the members.
- Different compilers or compiler settings may result in different amounts of padding.
- The compiler adds padding between members or at the end of the structure to ensure that the next member starts at an aligned address.

An aligned address refers to a memory address that is a multiple of a particular value, often dictated by the size of a data type. In the context of memory alignment in computer architecture, the alignment requirement is typically expressed in terms of the size of the data type. An address is considered aligned if it is divisible by the size of the data type, meaning there is no remainder when dividing the address by the size.

For example:

- If the alignment requirement is 4 bytes, then addresses like 0x00, 0x04, 0x08, 0x0C, etc., are considered aligned because they are multiples of 4.
- If the alignment requirement is 8 bytes, then addresses like 0x00, 0x08, 0x10, 0x18, etc., are considered aligned because they are multiples of 8.

Memory alignment is crucial for efficient memory access, especially for certain processors that may have performance penalties associated with accessing misaligned data. When the compiler adds padding between structure members, it aims to ensure

that each member starts at an aligned address according to the requirements of the architecture, which helps optimise memory access.

- It's important to note that the compiler may not always add padding. In cases where the memory location where the previous variable ends is already divisible by (aligned to) the `sizeof` the next variable, the compiler may choose not to add padding bytes. The decision depends on the alignment requirements and the specific architecture.

4. Example:

C++

```
struct Example {  
    char a;      // Takes 1 byte  
    int b;       // Takes 4 bytes  
    double c;    // Takes 8 bytes  
};  
cout<<sizeof(Example)<<endl;
```

Output

16

- Without padding, the total size of `Example` would be 13 bytes (1 + 4 + 8).
- However, the compiler may add 3 bytes of padding after `char a` to ensure that `int b` starts at an address divisible by 4. So, the actual size becomes 16 bytes.

5. Compiler Directives for Controlling Padding:

- Some compilers provide directives or pragmas to control or disable padding. For example, `#pragma pack` in C and C++ can be used to adjust the alignment of structures.
- However, changing packing settings should be done carefully, considering both alignment requirements and performance implications.

6. Considerations:

- While padding ensures proper alignment, it may lead to increased memory usage. This trade-off between memory efficiency and access speed is important to consider in performance-critical applications.

Understanding padding is essential for writing efficient and portable C++ code, especially in scenarios where memory layout and access patterns play a crucial role in the performance of the application.

Memory Alignment

not in syllabus

Memory alignment refers to the requirement that data should be stored in memory at addresses that are multiples of a particular size, which is determined by the computer architecture. Proper alignment is crucial for efficient memory access and performance, especially in systems where misaligned data can result in additional processing overhead or even errors.

Here are key points about memory alignment:

1. Processor Architecture:

- Different processors may have different alignment requirements. For example, a processor might require that a 4-byte integer starts at an address that is a multiple of 4.

2. Performance Impact:

- Misaligned data can lead to performance penalties. Some processors may require multiple memory accesses or perform additional processing to fetch misaligned data, which can slow down program execution.

3. Data Types:

- The alignment requirements often depend on the size and type of data. Common alignments include 1-byte, 2-byte, 4-byte, and 8-byte alignments for char, short, int, and double data types, respectively.

4. Compiler Decisions:

- Compilers play a significant role in achieving proper alignment. They may add padding between structure members or at the end of a structure to align data correctly based on the target architecture.

5. Padding:

- Padding involves adding extra bytes to ensure that the next data member in a structure starts at an aligned address. The size of the padding is determined by the alignment requirement of the specific data type.

6. Pragma Pack:

- Some compilers provide directives or pragmas (such as `#pragma pack` in C and C++) that allow programmers to control or disable padding for structures. However, altering packing settings should be done with caution, considering both alignment and performance.

7. Memory Access Efficiency:

- Aligned memory access allows the processor to fetch data in a single operation, improving efficiency. Misaligned data may require multiple fetch operations, impacting performance.

Understanding and managing memory alignment is essential for writing efficient and performant code, particularly in systems where memory access speed is critical, such as embedded systems, real-time applications, and high-performance computing environments.

C++

```
struct book{
char name[10];
int no_pages;
float price;
};
struct book e1[3];

for(int i=0; i<3; i++){
cout<<"Enter name of book "<<i+1<<" : ";
cin>>e1[i].name;
cout<<"Enter no. of pages in book "<<i+1<<" : ";
cin>>e1[i].no_pages;
cout<<"Enter price of book "<<i+1<<" : ";
cin>>e1[i].price;
}

for(int i=0; i<3; i++){
cout<<"Name of book"<<i+1<<" : ";
cout<<e1[i].name<<endl;
cout<<"No. of pages in book "<<i+1<<" : ";
cout<<e1[i].no_pages<<endl;
cout<<"Price of book "<<i+1<<" : ";
cout<<e1[i].price<<endl;
}
```

Output

```
Enter name of book 1 : Lotus
Enter no. of pages in book 1 : 500
Enter price of book 1 : 650
Enter name of book 2 : Lily
Enter no. of pages in book 2 : 400
Enter price of book 2 : 650
Enter name of book 3 : Rose
Enter no. of pages in book 3 : 300
Enter price of book 3 : 550

Name of book1 : Lotus
No. of pages in book 1 : 500
Price of book 1 : 650
Name of book2 : Lily
```

No. of pages in book 2 : 400
Price of book 2 : 650
Name of book3 : Rose
No. of pages in book 3 : 300
Price of book 3 : 550

C++

```
struct book{
char name[10]; // TAKES 10 BYTES OF MEMORY
int no_pages; // TAKES 4 BYTES OF MEMORY
float price; // TAKES 4 BYTES OF MEMORY
};
struct book e1[3]; // SHALL TAKE 20*3 = 60 BYTES OF CONTIGUOUS MEMORY, 10
BYTES OF CHAR ARRAY, 4 BYTES OF INT, 4 BYTES OF FLOAT AND 2 BYTES FOR
PADDING

cout<<"sizeof(e1) : "<<sizeof(e1)<<endl;
cout<<"sizeof(e1[0]) : "<<sizeof(e1[0])<<endl;
cout<<endl;
cout<<"&e1[0] : "<<&e1[0]<<endl;
cout<<"&e1[1] : "<<&e1[1]<<endl;
cout<<"&e1[2] : "<<&e1[2]<<endl;
```

Output

```
sizeof(e1) : 60
sizeof(e1[0]) : 20

&e1[0] : 0x7fffffffdd670
&e1[1] : 0x7fffffffdd684
&e1[2] : 0x7fffffffdd698
```

C++

```
struct book{
int i;
float j;
};

struct test{
int z;
struct book e1;
};

struct test b1;
```

```
b1.e1.i = 50;
b1.e1.j = 80;

cout<<b1.e1.i<<endl<<b1.e1.j;
```

Output

```
50
80
```

C++

```
struct book{
int i;
float j;
} e1 = {20,30}, e2;
e2 = e1;
cout<<e2.i<<endl<<e2.j;
```

Output

```
20
30
```

C++

```
struct book {
int i; // 4 BYTES
float j; // 4 BYTES
// struct book b1; SYNTAX ERROR
struct book *b1; // A POINTER THAT STORES THE ADDRESS OF A VARIABLE OF
`struct book` TYPE, CAN DO THIS, ADDRESS IS OF 8 BYTES
// THIS IS A SELF REFERENTIAL STRUCTURE
} e1; // EXPECTED 16 BYTES

cout<<sizeof(e1);
```

Output

```
16
```

C++

```
struct book {
int i; // 4 BYTES
```

```

float j; // 4 BYTES
} e1 = {10, 30}; // 8 BYTES

struct show {
int z; // 4 BYTES
struct book p {40, 90}; // 8 BYTES
} k; // 12 BYTES
// CAN DO THIS

cout<<"sizeof(e1) : "<<sizeof(e1)<<endl; // 8
cout<<"sizeof(k) : "<<sizeof(k)<<endl; // 12
cout<<endl;
cout<<"e1.i : "<<e1.i<<endl; // 10
cout<<"k.z : "<<k.z<<endl; // GARBAGE VALUE
cout<<"k.p.i : "<<k.p.i<<endl; // 40
cout<<"k.p.j. : "<<k.p.j<<endl; // 90

```

Output

```

sizeof(e1) : 8
sizeof(k) : 12

e1.i : 10
k.z : 0
k.p.i : 40
k.p.j. : 90

```

C++

```

struct book {
int i; // 4 BYTES
float j; // 4 BYTES
} e1 = {10, 30}, *p;
p = &e1;

cout<<"p->i : "<<p->i<<endl; // 10
cout<<"p->j : "<<p->j<<endl; // 30
p->i = 50;
cout<<endl;
cout<<"p : "<<p<<endl;
cout<<"&p : "<<&p<<endl;
// cout<<e1; SYNTAX ERROR
cout<<"&e1 : "<<&e1<<endl;

```

Output

```
p->i : 10
```

```
p->j : 30
```

```
p : 0x7fffffffed6b0
```

```
&p : 0x7fffffffed6a8
```

```
&e1 : 0x7fffffffed6b0
```

Self-Referential Structures

A self-referential structure in programming is a structure that contains a member pointing to another instance of the same type of structure. This allows for the creation of linked data structures, such as linked lists, trees, or graphs, where each element refers to the next or related elements of the same kind.

As you will see in object oriented programming paradigm you can also create self referential classes; a well-designed program that implements linked data structures will demonstrate the effective use of classes and structures, showcasing principles of encapsulation, data abstraction, and modular design.

In the C programming language, classes do not exist; hence, it does not inherently support object-oriented programming paradigm. However, C can still implement some OOP concepts, such as encapsulation and abstraction, through the use of structures and function pointers

C++

```
struct node{
int i; // 4 BYTES
struct node *p; // 8 BYTES
} k, m; // 12 BYTES EACH, `k` AND `m` MAY OR MAY NOT BE CONTIGUOUS

k.i = 10;
m.i = 20;
k.p = &m;
m.p = &k;

cout<<"m.p -> i : "<<m.p -> i<<endl; // 10
cout<<"k.p -> i : "<<k.p -> i<<endl; // 20

m.p -> i = 80; // SAME AS `k.i = 80;`
k.p -> i = 90; // SAME AS `m.i = 80;`
cout<<"k.i : "<<k.i<<endl; // 80
cout<<"m.i : "<<m.i<<endl; // 90
cout<<"&k : "<<&k<<endl;
cout<<"m.p : "<<m.p<<endl;
cout<<"m.p -> p : "<<m.p -> p<<endl;
cout<<"k.p -> p : "<<k.p -> p<<endl;
```

Question

Write a program to create a linked list with four points.

Solution

C++

```

struct node{
int i;  // 4 BYTES
struct node *p;  // 8 BYTES
} k, m, z, x, y;

k.i = 1;
m.i = 2;
z.i = 3;
x.i = 4;
k.p = &m;
m.p = &z;
z.p = &x;
x.p = &y;

cout<<"&k : "<<&k<<endl;
cout<<"k.p : "<<k.p<<endl;
cout<<"&m : "<<&m<<endl;
cout<<"m.p : "<<m.p<<endl;
cout<<"&z : "<<&z<<endl;
cout<<"z.p : "<<z.p<<endl;
cout<<"&x : "<<&x<<endl;
cout<<"x.p : "<<x.p<<endl;
cout<<"&y : "<<&y<<endl;

```

Output

```

&k : 0x7fffffff640
k.p : 0x7fffffff650
&m : 0x7fffffff650
m.p : 0x7fffffff660
&z : 0x7fffffff660
z.p : 0x7fffffff670
&x : 0x7fffffff670
x.p : 0x7fffffff680
&y : 0x7fffffff680

```

Question

Write a program to create a circular linked list with four points.

Solution

C++

```

struct node{
int i;  // 4 BYTES
struct node *p;  // 8 BYTES
} k, m, z, x;

```

```

k.i = 1;
m.i = 2;
z.i = 3;
x.i = 4;
k.p = &m;
m.p = &z;
z.p = &x;
x.p = &k;

cout<<"&k : "<<&k<<endl;
cout<<"k.p : "<<k.p<<endl;
cout<<"&m : "<<&m<<endl;
cout<<"m.p : "<<m.p<<endl;
cout<<"&z : "<<&z<<endl;
cout<<"z.p : "<<z.p<<endl;
cout<<"&x : "<<&x<<endl;
cout<<"x.p : "<<x.p<<endl;

```

Output

```

&k : 0x7fffffff670
k.p : 0x7fffffff680
&m : 0x7fffffff680
m.p : 0x7fffffff690
&z : 0x7fffffff690
z.p : 0x7fffffff6a0
&x : 0x7fffffff6a0
x.p : 0x7fffffff670

```

C++

```

int main(){
void show(int); // FUNCTION PROTOTYPE DECLARATION
show(10);
return 0;
}

void show(int z){ // FUNCTION DEFINITION
cout<<z<<endl;
}

```

Output

```

10

```

C++

```

struct book{
int i;
int j;
};
int main(){
void show(struct book);
struct book k = {10, 20};
show(k);
return 0;
}

void show(struct book m){
cout<<"m.i : "<<m.i<<endl;
cout<<"m.j : "<<m.j<<endl;
}

```

Output

```

m.i : 10
m.j : 20

```

C++

```

struct book{
int i;
int j;
};
int main(){
void show(struct book);
struct book k = {10, 20};
show(k);
cout<<"k.i : "<<k.i<<endl;
return 0;
}

void show(struct book m){
m.i = 30;
cout<<"m.i : "<<m.i<<endl;
}

```

Output

```

m.i : 30
k.i : 10

```

C++

```
struct book{
int i;
int j;
};
int main(){
void show(struct book *);
struct book k = {10, 20};
show(&k);
cout<<"k.i : "<<k.i<<endl;
cout<<"k.j : "<<k.j<<endl;
return 0;
}

void show(struct book *m){
m->i = 50;
m->j = 80;
cout<<"m->i : "<<m->i<<endl;
cout<<"m->j : "<<m->j<<endl;
}
```

Nested Structures

A nested structure is defined when one of the members of a structure is itself a structure.

To access members of a nested structure, you use the dot operator (.) for each level of the structure.

C++

```
struct Address {
    char street[100];
    char city[50];
    char state[50];
    int zip; };

struct Employee {
    char name[100];
    int empID;
    struct Address addr; // Nested structure
};
```

C++

```
struct book {
    int i = 10;
    struct{
        int j = 90;
    }b;
    }a;

cout<<"a.i : "<<a.i<<endl;
cout<<"a.b.i : "<<a.b.j<<endl;
```

Union

A union is a user-defined data type that allows different data types to be stored in the same memory location. Unlike structures, where each member has its own memory space, union members share the same memory space. This means that only one member of the union can be active (contain a value) at any given time. Changing the value of one member affects the values of the other members because they share the same memory location.

- Union members share the same memory location.
- Only one member can have a value at a time.
- The size of a union is determined by the size of its largest member.
- Accessing a union member retrieves the data based on the last stored value.

syntax

C++

```
union name_of_union{  
  declaration of variables  
};  
union name_of_union variable_representing_union;
```

C++

```
union name_of_union{  
  declaration of variables  
} variable_representing_union;
```

1. Memory equal to the largest member of the union is allotted to the union.
2. The starting memory address of a union and all its members is the same.

C++

```
union book{  
  char ch;  
  float j;  
  int k;  
}z;  
  
cout<<"sizeof(z) : "<<sizeof(z)<<endl;  
  
z.k = 6;  // ASSIGNING A VALUE TO A VARIABLE `k` OF THE UNION `book`,  
OTHER TWO SHALL NOT HAVE A SIGNIFICANT VALUE
```

```

cout<<endl<<"z.k = 6"<<endl;
cout<<"z.k : "<<z.k<<endl;
cout<<"z.j : "<<z.j<<endl; // EXPECTED GARBAGE VALUE
cout<<"z.ch : "<<z.ch<<endl; // EXPECTED GARBAGE VALUE
z.j = 2.5; // ASSIGNING A VALUE TO A VARIABLE `k` OF THE UNION `book`,
OTHER TWO SHALL NOT HAVE A SIGNIFICANT VALUE
cout<<endl<<"z.j = 2.5"<<endl;
cout<<"z.k : "<<z.k<<endl; // EXPECTED GARBAGE VALUE
cout<<"z.j : "<<z.j<<endl;
cout<<"z.ch : "<<z.ch<<endl; // EXPECTED GARBAGE VALUE
z.ch = 'A'; // ASSIGNING A VALUE TO A VARIABLE `k` OF THE UNION `book`,
OTHER TWO SHALL NOT HAVE A SIGNIFICANT VALUE
cout<<endl<<"z.ch = 'A'"<<endl;
cout<<"z.k : "<<z.k<<endl; // EXPECTED GARBAGE VALUE
cout<<"z.j : "<<z.j<<endl; // EXPECTED GARBAGE VALUE
cout<<"z.ch : "<<z.ch<<endl;

```

Output

```

sizeof(z) : 4

z.k = 6
z.k : 6
z.j : 8.40779e-45
z.ch :

z.j = 2.5
z.k : 1075838976
z.j : 2.5
z.ch :

z.ch = 'A'
z.k : 1075839041
z.j : 2.50002
z.ch : A

```

C++

```

union book{
char ch;
float j;
int k;
}z;

char *cptr = &z.ch;
float *jptr = &z.j;
int *kptr = &z.k;

```

```

cout<<"sizeof(z) : "<<sizeof(z)<<endl;
cout<<"&z : "<<&z<<endl;
cout<<"&z.k : "<<&z.k<<endl;
cout<<"&z.j : "<<&z.j<<endl;
cout<<"&z.ch : "<<(int *)&z.ch<<endl; // HAVE TO TYPE CAST BECAUSE
`&z.ch` RETURNS A POINTER TO `char` DATA TYPE, BUT `cout` EXPECTS A
NUMERIC TYPE POINTER

```

Output

```

sizeof(z) : 4
&z : 0x7fffffff69c
&z.k : 0x7fffffff69c
&z.j : 0x7fffffff69c
&z.ch : 0x7fffffff69c

```

C++

```

union book{
char ch;
float j;
int k;
}z;

char *cptr = &z.ch;
float *jptr = &z.j;
int *kptr = &z.k;

z.k = 6;
cout<<endl<<"z.k = 6"<<endl;
cout<<"z.k : "<<z.k<<endl;
cout<<"z.j : "<<z.j<<endl;
cout<<"z.ch : "<<z.ch<<endl;
cout<<"*kptr : "<<*kptr<<endl;
cout<<"*jptr : "<<*jptr<<endl;
cout<<"*cptr : "<<*cptr<<endl;

z.j = 2.5;
cout<<endl<<"z.j = 2.5"<<endl;
cout<<"z.k : "<<z.k<<endl;
cout<<"z.j : "<<z.j<<endl;
cout<<"z.ch : "<<z.ch<<endl;
cout<<"*kptr : "<<*kptr<<endl;
cout<<"*jptr : "<<*jptr<<endl;
cout<<"*cptr : "<<*cptr<<endl;

z.ch = 'A';

```

```
cout<<endl<<"z.ch = 'A'"<<endl;
cout<<"z.k : "<<z.k<<endl;
cout<<"z.j : "<<z.j<<endl;
cout<<"z.ch : "<<z.ch<<endl;
cout<<"*kptr : "<<*kptr<<endl;
cout<<"*jptr : "<<*jptr<<endl;
cout<<"*cptr : "<<*cptr<<endl;
```

Output

```
z.k = 6
z.k : 6
z.j : 8.40779e-45
z.ch :
*kptr : 6
*jptr : 8.40779e-45
*cptr :

z.j = 2.5
z.k : 1075838976
z.j : 2.5
z.ch :
*kptr : 1075838976
*jptr : 2.5
*cptr :

z.ch = 'A'
z.k : 1075839041
z.j : 2.50002
z.ch : A
*kptr : 1075839041
*jptr : 2.50002
*cptr : A
```

Strings

1. A string is a sequence of characters.

A character in C++ is a byte sized data-type capable of storing 256 distinct values from 0 to 255.

2. `string` provides dynamic size adjustment, convenient member functions, and automatic memory management.
3. When initialising a `char` variable with a specific character, it is necessary to enclose the character in single quotes (`' '`).
4. When declaring or initialising a string, it is essential to enclose the string within double quotation marks (`" "`).
5. Consider either creating a character array (C-Style String), where each character is stored at a specific index in the array, or utilising the built-in string data type in C++ (string object).
6. Storing characters in a character array assigns a unique address to each letter, allowing them to be represented by distinct variables. This facilitates various tasks and simplifies many operations.
7. After the input of a list, string, or word concludes, the compiler automatically appends a null termination `\0` to the array. This indicates the end of the input. When using `cout` , it prints from the beginning of the array until the null termination, signifying the completion of the input.
8. A string object in C++ lacks null termination.

As compiler is platform dependent, the implementation of appending a null termination in a `string` object is also platform dependent. However, transgression of this is very rarely seen.

9. The functions `strcmp` , `strcat` , `strlen` , etc. operate on character arrays (C-Strings) rather than string variables in C++ , as strings lack null termination.

C-Style Strings

A C-style string is a sequence of characters stored in an array of type `char` in the C programming language. C does not have a built-in string data type, so strings are typically represented as arrays of characters terminated by a null character (`'\0'`). This null character is used to signify the end of the string.

Here's how you declare and initialise a C-style string:

C++

```
char myString[] = "Hello, World!";
```

In this example, `myString` is a C-style string containing the characters "Hello, World!". The size of the array is automatically determined based on the length of the string literal, including the null character.

C-style strings have several characteristics:

1. **Null Termination:** The string is terminated with a null character (`'\0'`), which marks the end of the string. This allows functions to determine the length of the string.
2. **Mutable:** The contents of a C-style string can be modified, but you need to be careful not to exceed the allocated space and ensure that the null character is maintained.
3. **Array of Characters:** A C-style string is essentially an array of characters, and you can access individual characters using array notation (`myString[0]` , `myString[1]` , etc.).
4. **Static Size:** The size of the array must be large enough to accommodate the string and the null character. It's your responsibility to ensure that the array is large enough to hold the intended string.

Here are some common string functions in C that work with C-style strings:

- `strlen()` : Calculates the length of a string.
- `strcpy()` : Copies one string to another.
- `strcat()` : Concatenates two strings.
- `strcmp()` : Compares two strings.

Remember to handle C-style strings with care to avoid buffer overflows and unintended behaviour. C++ introduced the `std::string` class (used for creating string objects) to provide a safer and more convenient alternative to C-style strings.

C++

```
// char ch = 'abc'; SYNTAX ERROR  
char ch = 'a'; // CAN DO
```

C++

```
string ch = "abcd";  
cout<<ch;
```

Output

abcd

C++

```
string ch[3]={"abc", "def", "mits"};  
cout<<ch[1];
```

Output

def

C++

```
char ch[20]="abcd";  
cout<<ch;
```

Output

abcd

C++

```
char ch[20];  
cout<<"Enter ch : ";  
cin>>ch;  
cout<<"ch : "<<ch;
```

Output

```
Enter ch : abcd  
ch : abcd
```

```
char ch[20];
cout<<"Enter ch : ";
cin>>ch;
cout<<"ch (w/o for loop): "<<ch<<endl;
cout<<"ch (w/ for loop): ";
for(int i = 0; i < 20; i++){
cout<<ch[i];
}
```

```
Enter ch : abcd
ch (w/o for loop): abcd
ch (w/ for loop): abcd@*~
```

```
char ch[20];
cout<<"Enter ch : ";
cin>>ch;
cout<<"ch (w/o for loop): "<<ch<<endl;
cout<<"ch (w/ for loop): ";
for(int i = 0; ch[i] != '\0'; i++){
cout<<ch[i];
}
```

```
Enter ch : abcd
ch (w/o for loop): abcd
ch (w/ for loop): abcd
```

```
// #include <string.h>
char ch1[20];
char ch2[20];

cout<<"Enter string ch1 : ";
cin>>ch1;
cout<<"Enter string ch2 : ";
cin>>ch2;
cout<<endl;
cout<<"ch1 : "<<ch1<<endl;
```

```
cout<<"ch2 : "<<ch2<<endl;
cout<<"strlen(ch1) : "<<strlen(ch1)<<endl;
cout<<"strlen(ch2) : "<<strlen(ch2)<<endl;
cout<<"strcat(ch1,ch2) : "<<strcat(ch1,ch2)<<endl;
cout<<"strcmp(ch1,ch2) : "<<strcmp(ch1,ch2)<<endl;
```

Output

```
Enter string ch1 : sd
Enter string ch2 : ds

ch1 : sd
ch2 : ds
strlen(ch1) : 2
strlen(ch2) : 2
strcat(ch1,ch2) : sdds
strcmp(ch1,ch2) : 15
```

Pre-Defined String Functions

The `string.h` header file provides a set of functions for working with C-style strings (character arrays).

1. `strlen`:

- **Description:** Calculates the length of a C-style string.
- **Syntax:** `size_t strlen(const char* str);`

2. `strcpy`:

- **Description:** Copies a C-style string from source to destination.
- **Syntax:** `char* strcpy(char* dest, const char* src);`

3. `strcat`:

- **Description:** Concatenates (appends) the contents of one C-style string to another.
- **Syntax:** `char* strcat(char* dest, const char* src);`

4. `strcmp`:

- **Description:** Compares two C-style strings lexicographically.
- **Syntax:** `int strcmp(const char* str1, const char* str2);`

`strcmp` returns an integer value representing the lexicographical comparison result:

- Returns 0 if the strings are equal.
- Returns a positive value if `str1` is lexicographically greater than `str2`.
- Returns a negative value if `str1` is lexicographically smaller than `str2`.
- The `strcmp` function compares the characters in `str1` and `str2` lexicographically.
- It stops the comparison when a differing character is encountered or when the null terminator (`'\0'`) is reached.
- The returned integer value indicates the relationship between the strings based on lexicographical order.
- This function is case-sensitive, meaning uppercase and lowercase letters are treated differently.
- For case-insensitive comparison, you might consider using `stricmp` (Windows) or `strcasecmp` (Unix-like systems), which are non-standard but commonly available

5. `strncmp`:

- **Description:** Compares a specified number of characters of two C-style strings.
- **Syntax:** `int strncmp(const char* str1, const char* str2, size_t num);`

6. `strchr`:

- **Description:** Finds the first occurrence of a character in a C-style string.
- **Syntax:** `char* strchr(const char* str, int character);`

7. `strtok` :

- **Description:** Splits a C-style string into tokens based on specified delimiters.
- **Syntax:** `char* strtok(char* str, const char* delimiters);`

8. `strstr` :

- **Description:** Finds the first occurrence of a substring in a C-style string.
- **Syntax:** `char* strstr(const char* str1, const char* str2);`

9. `memcpy` :

- **Description:** Copies a specified number of bytes from one memory location to another.
- **Syntax:** `void* memcpy(void* dest, const void* src, size_t num);`

10. `memset` :

- **Description:** Sets a specified number of bytes in a memory block to a particular value.
- **Syntax:** `void* memset(void* block, int value, size_t size);`

These functions provide essential operations for manipulating and working with C-style strings. Keep in mind that C++ also offers the `std::string` class in the `<string>` header, which provides a more modern and convenient way to handle strings with dynamic memory management and a rich set of member functions.

C++

```
// #include <string.h>
char ch[20] = "mits";
char ch1[20];
cout<<ch<<endl;
cout<<strlen(ch)<<endl;
cout<<strrev(ch)<<endl;
cout<<strupr(ch)<<endl;
cout<<strlwr(ch)<<endl;
strcpy(ch1, ch); // SOURCE = ch , DESTINATION = ch1
cout<<ch1<<endl;
cout<<strcmp(ch, ch1)<<endl;
cout<<strcmpi(ch, ch1)<<endl; // NOT CASE SENSITIVE
```

Output

```
mits
4
stim
MITS
mits
mits
0
0
```

C++

```
char ch[] = "mits";  
char ch1[] = "t";  
  
cout<<strstr(ch, ch1); // FINDS `ch1` IN `ch` AND PRINTS FROM FOUND INDEX  
TILL LAST
```

Output

```
ts
```

Question

Write a program to perform string copy, string comparison, string reversal without using string.h header file.

Solution

C++

```
char ch1[50];  
char ch2[50];  
int i;  
cout<<"Enter string 1 : ";  
cin>>ch1;  
cout<<"Enter string 2 : ";  
cin>>ch2;  
  
cout<<endl<<"String Comparision"<<endl;  
bool str_same = 1;  
for (i = 0; ((ch1[i] != '\0')&&(ch2[i] != '\0')); i++){  
    if (ch1[i] != ch2[i])  
        str_same = 0;  
}  
if (str_same)  
    cout<<"Strings are same"<<endl;  
else  
    cout<<"Strings are not same"<<endl;  
  
cout<<endl<<"String Copy"<<endl;  
char ch3[50];  
for (i = 0; ch1[i] != '\0'; i++)  
    ch3[i] = ch1[i];  
int str_len = i;  
ch3[i] = ch1[i]; // ch3[i] = '\0';  
cout<<"Copied string : "<<ch3<<endl;
```

```
cout<<endl<<"String Reversal"<<endl;
char rev_str[50];
for(i = 0; i < str_len; i++)
    rev_str[i] = chl[str_len - i - 1];
rev_str[i] = chl[str_len];
cout<<"Reversed String : "<<rev_str<<endl;
```

Output

```
Enter string 1 : suresh
Enter string 2 : peoneda
```

```
String Comparision
Strings are not same
```

```
String Copy
Copied string : suresh
```

```
String Reversal
Reversed String : hserus
```

Gets & Puts Functions

not in syllabus

`gets` and `puts` are functions used for reading and writing strings. However, it's important to note that these functions are considered unsafe due to their vulnerability to buffer overflow attacks. It's generally recommended to use safer alternatives like `fgets` and `fputs`.

1. `gets` Function:

- **Declaration:** `char* gets(char* str);`
- **Purpose:** Reads a line from the standard input (usually the keyboard) and stores it in the string pointed to by `str`. It continues reading until a newline character or the end-of-file (EOF) is encountered.
- **Issues:** The major problem with `gets` is that it does not check the size of the destination buffer, making it susceptible to buffer overflow.

C++

```
char buffer[50];
cout << "Enter a string: ";
gets(buffer); // Unsafe, can lead to buffer overflow
```

The `gets` function has been deprecated and removed from the C++ standard due to its unsafe nature and susceptibility to buffer overflow issues.

2. `puts` Function:

- **Declaration:** `int puts(const char* str);`
- **Purpose:** Writes a string (null-terminated) to the standard output (usually the console). It appends a newline character after the string.
- **Return Value:** Returns a non-negative value on success, and `EOF` on failure.

C++

```
const char* message = "Hello, World!";
puts(message);
```

Output

Hello, World!

Avoiding Gets & Puts Functions

not in syllabus

The `gets` and `puts` functions in C++ are considered unsafe due to their inherent lack of bounds checking, making them susceptible to buffer overflow vulnerabilities. Here's a brief explanation of the security vulnerabilities associated with these functions:

1. Buffer Overflow:

- `gets` reads characters from the standard input and stores them in the specified buffer until a newline character is encountered or the buffer is full.
- There is no mechanism to limit the number of characters read, which can lead to a buffer overflow if the input exceeds the size of the buffer.
- An attacker can exploit this vulnerability by providing input that is larger than the buffer size, overwriting adjacent memory and potentially executing malicious code.

2. No Bounds Checking:

- Both `gets` and `puts` lack bounds checking. They assume that the provided buffer is large enough to hold the input, which can lead to unpredictable behavior when working with insufficiently sized buffers.
- This lack of bounds checking makes these functions error-prone and prone to exploitation.

3. Security Best Practices:

- To mitigate the risks associated with buffer overflows, it's recommended to use safer alternatives like `fgets` for input and `fputs` for output.
- `fgets` allows you to specify the maximum number of characters to read, preventing buffer overflows.
- Similarly, `fputs` is a safer choice for output, as it doesn't rely on null-terminated strings and allows better control over what is written.

Here's an example of using `fgets` for safer input:

C++

```
char buffer[10];

cout << "Enter text: ";
fgets(buffer, sizeof(buffer), stdin);

cout << "You entered: " << buffer << endl;
```

By using functions that perform bounds checking, you can enhance the security of your code and reduce the risk of vulnerabilities associated with buffer overflows.

Buffer Overflows

not in syllabus

Buffer overflow is a security vulnerability that occurs when a program writes more data to a block of memory, or buffer, than it was allocated to hold. This overflow can lead to unexpected behaviour, crashes, and, more critically, exploitation by attackers.

Here's a basic explanation of how buffer overflow works:

1. Memory Buffers:

- In programming, buffers are regions of memory set aside to hold data. They have a fixed size determined during the program's design or runtime.

2. Data Input:

- If the program accepts external input (like user input), it may store that input in a buffer.

3. Writing Data:

- When a program writes data to a buffer, it assumes that the data will fit within the allocated space.

4. Exceeding Buffer Limits (No Bounds Checking):

- If the program does not check the size of the input against the size of the buffer, and the input is larger than the buffer can hold, then the program writes more data than the buffer can hold, and the excess data overflows into adjacent memory locations.

5. Security Implications:

- If an attacker can control the data written to a buffer and cause an overflow, they might overwrite critical program data, alter the program's flow, or inject malicious code.

6. Exploitation:

- An attacker can craft input in such a way that it overflows the buffer with specific values. By doing so, they may overwrite critical data structures, manipulate program execution, or even inject and execute their own code. When the compromised program returns from the function, it may execute the attacker's injected code.

7. Consequences:

- Buffer overflows can lead to crashes, data corruption, unauthorised access, or the execution of arbitrary code. This poses a severe threat to the security and stability of a system.

Buffer overflows are serious security flaws because they can lead to unpredictable behaviour, crashes, and potentially allow attackers to compromise the security of a system. Modern programming practices, such as using secure functions and incorporating bounds

checking, aim to prevent buffer overflow vulnerabilities. Additionally, various security mechanisms, like address space layout randomisation (ASLR) and Data Execution Prevention (DEP), are employed to make it more difficult for attackers to exploit these vulnerabilities.

C++

```
// Allocate a buffer with a size of 10
char buffer[10];

// Get input from the user
cout<<"Enter a string: ";
cin>>buffer;

// Display the input
cout<<"You entered: "<<buffer<<endl;
```

Output

```
Enter a string: abcdefghijklmnopqrst
You entered: abcdefghijklmnopqrst
*** stack smashing detected ***: terminated
```

Output

```
Enter a string: abcdefghijklmnopqrst
You entered: abcdefghijklmnopqrst
Segmentation fault
```

Summary

The provided code attempts to read a string input from the user using `cin` and store it in a buffer of size 10. The outcomes differ based on the length of the entered string.

1. Output1:

- Input: "abcdefghijklmnopqrst" (length: 20)
- Output: "You entered: abcdefghijklmnopqrst"
- Additional Message: `*** stack smashing detected ***: terminated`
- This indicates a buffer overflow, where the input exceeds the buffer size, leading to a termination signal.

2. Output2:

- Input: "abcdefghijklmnopqrst" (length: 20)
- Output: "You entered: abcdefghijklmnopqrst"
- Additional Message: `Segmentation fault`

- While the input is shorter and doesn't trigger the stack smashing detection, a segmentation fault occurs, likely due to memory access issues.
3. The code is vulnerable to buffer overflow because it doesn't ensure that the input fits within the allocated buffer size.
 4. Both cases demonstrate issues with memory access, resulting in termination signals.
 5. To address this, it's recommended to use safer alternatives like `std::cin.getline` or `std::getline` to handle string inputs more securely and prevent buffer overflows.

C++

```
// #include <string.h>

void vulnerableFunction(const char* input) {
    char buffer[10];
    strcpy(buffer, input); // This line is vulnerable to buffer overflow
    cout<<"Buffer content: "<<buffer<<endl;
}

int main() {
    const char maliciousInput[] = "12345678901234567890"; // This input is
    longer than the buffer size
    vulnerableFunction(maliciousInput);
    return 0;
}
```

Output

```
Buffer content: 12345678901234567890
*** stack smashing detected ***: terminated
```

Output

```
Buffer content: 12345678901234567890
Segmentation fault
```

Summary

1. `vulnerableFunction` has a buffer of size 10 (`char buffer[10]`).
2. The `strcpy` function is used to copy the content of the `input` parameter into the buffer.
3. The `main` function creates an array `maliciousInput` with a length of 20, which is longer than the buffer in `vulnerableFunction`.

When the `vulnerableFunction` is called with `maliciousInput`, it causes a buffer overflow because the input is larger than the buffer size. This can lead to unpredictable behaviour,

crashes, and potentially exploitable security vulnerabilities.

In real-world scenarios, attackers may carefully craft inputs to overwrite critical data, control program flow, or inject malicious code into the program. To prevent such vulnerabilities, developers should use secure coding practices, bounds checking, and avoid using unsafe functions like `strcpy`.

Note

The discrepancy in the output messages between different environments is likely due to variations in compiler settings and runtime environments.

In both cases, the program is exhibiting undefined behaviour due to a buffer overflow. The `strcpy` function is copying more data into the `buffer` than it can accommodate, leading to memory corruption.

The message `*** stack smashing detected ***: terminated` typically indicates that a stack-based buffer overflow has been detected by a compiler or runtime mechanism. It suggests that there are protections in place to detect such issues.

The `Segmentation fault` message indicates that the program is trying to access a restricted area of memory (likely due to the buffer overflow), leading to termination.

In both scenarios, the program is not behaving correctly, and the behaviour can vary based on the compiler, compiler settings, and runtime environment. It's essential to avoid buffer overflows in real-world code to ensure robust and secure programs.

Recursion

Recursion is a programming concept where a function calls itself in order to solve a problem. In a recursive function, the problem is divided into sub-problems, and each sub-problem is solved by applying the same algorithm. The recursion continues until a base case is reached, at which point the function stops calling itself and returns a result.

Key components of a recursive function:

1. **Base Case (Terminating Condition):** This is the condition that determines when the recursion should stop. It represents the simplest form of the problem that can be solved directly. Without a base case, the recursion would continue indefinitely.

Terminating condition is necessary in recursion.

2. **Recursive Case:** This is the part of the function where it calls itself with a modified version of the original problem. The recursion should lead towards the base case, ensuring that progress is made towards a solution.
3. **Input Modification:** Each recursive call should modify the input parameters in a way that brings the function closer to the base case. This ensures that the problem size decreases with each recursion.

C++

```
int show(){
    show();
}
```

Question

Write a program to calculate factorial of a number using recursion.

Solution

C++

```
int main(){
    int fact(int);
    int result;
    int m;
    cout<<"Enter a number : ";
    cin>>m;
    result=fact(m);
    cout<<"Factorial is : "<<result<<endl;

    return 0;
}
```

```
int fact(int n){  
  
    if (n == 1)  
        return 1;  
  
    else  
        return n * fact(n-1);  
}
```

Question

Write a program to add numbers up-to n using recursion.

Solution

C++

```
int main(){  
    int add(int);  
    int m;  
    cout<<"Enter a number : ";  
    cin>>m;  
    cout<<"Sum of numbers upto "<<m<<" is : "<<add(m)<<endl;  
}  
  
int add(int n){  
    if (n == 0)  
        return 0;  
  
    else  
        return n + add(n-1);  
}
```

Function Returning Addresses

In C++, a function can return an address by declaring its return type as a pointer. This is useful when you want a function to perform some computation and return a memory address, typically pointing to a dynamically allocated memory block or to a variable whose scope extends beyond the function. Returning an address allows the calling function to access the data or memory allocated by the called function.

C++

```
#include <iostream>
using namespace std;

int *show() {
    int z = 10;
    int *q;
    q = &z;
    cout << "q (in show fn) : " << q << endl;
    cout << "*q (in show fn) : " << *q << endl;
    return q;
}

int main() {
    int *p;
    p = show();
    cout << "p (in main fn) : " << p << endl;
    cout << "*p (in main fn) : " << *p << endl;
    return 0;
}
```

Output

```
q (in show fn) : 0x7fffffff68c
*q (in show fn) : 10
p (in main fn) : 0x7fffffff68c
*p (in main fn) : 32767
```

Explanation

In this code, the variable `z` is declared within the `show` function, and its address is assigned to the pointer `q`. However, since `z` is a local variable within the scope of the `show` function, its address becomes invalid once the function exits. Therefore, the pointer `p` in the `main` function now points to an invalid memory location, resulting in undefined behaviour when attempting to dereference `p`. Always be cautious when dealing with pointers and ensure that the memory being pointed to remains valid throughout its usage.

```

#include <iostream>
using namespace std;

int z = 10;

int main() {
    int *show();

    int *p;
    p = show();
    cout << "p (in main fn) : " << p << endl;
    cout << "*p (in main fn) : " << *p << endl;
    return 0;
}

int *show() {
    int *q;
    q = &z;
    cout << "q (in show fn) : " << q << endl;
    cout << "*q (in show fn) : " << *q << endl;
    return q;
}

```

Output

```

q (in show fn) : 0x7fffffff68c
*q (in show fn) : 10
p (in main fn) : 0x7fffffff68c
*p (in main fn) : 10

```

Explanation

In this modified code, the variable `z` is declared outside any function, making it a global variable. As a result, its scope extends throughout the entire program, and its address remains valid even after the `show` function exits. Therefore, when the pointer `p` in the `main` function is assigned the address of `z` through the `show` function, it points to a valid memory location, and dereferencing `p` correctly yields the value of `z`.

As per the updated guidelines of C++, we are not allowed to directly return the address of a local variable; instead, we need to create a pointer for it and then return the pointer. Attempting to directly return the address of a local variable results in a warning, indicating that it cannot be stored directly in another function's variable.

```
address of local variable 'z' returned [-Wreturn-local-addr]gcc
```

A reference returned by a function to a local variable of the said function is known as a dangling reference, as when the function returns (hence returning a reference to the calling function) all the local variables of the called function "vanish" at this point, leading to a reference at an undefined location

Therefore, before returning a reference we must ensure that the reference is beyond the scope of the called function

Algorithm

An algorithm is a systematic sequence of steps designed to perform a specific task on computers.

Characteristics of an Algorithm

1. Algorithms should be independent of any programming language.
2. An algorithm must have a finite number of steps, meaning it should eventually stop.
3. It can take zero or more inputs and produce one or more outputs.

Algorithm Performance Analysis

Algorithm performance is analysed based on two criteria:

1. **Time (Time Complexity)**
2. **Space (Space Complexity)**

Time Complexity of a Program (Running Time)

The running time of a program depends on factors such as:

1. Number of executable codes
2. Hardware (Processor)
3. Software (Operating System)
4. Number of inputs
5. Method used

Time Complexity of an Algorithm (Running Time)

The running time of an algorithm depends on factors like:

1. Number of executions
2. Number of inputs
3. Method used.

Asymptotic Notations

1. **Big Oh (Worst-case Time Complexity):** Denoted as $O()$.

2. **Omega (Best-case Time Complexity):** Denoted as $\Omega()$.
3. **Theta (Average-case Time Complexity):** Denoted as $\Theta()$.

Types of Complexity

1. **Logarithmic:** $O(\log(n))$
2. **Linear:** $O(n)$
3. **Polynomial:** $O(n^2, n^3)$
4. **Exponential:** $O(2^n, 3^n)$

Command Line Arguments

Command-line arguments offer a means for users to provide inputs to a program when it is run from the command line or terminal. These arguments are handled through the `main` function's parameters `argc` and `argv`.

In C++, command-line arguments provide a way to pass information to a program when it is run from the command line or terminal.

The `main` function in C++ can have two different forms:

1. Basic Form:

C++

```
int main() {  
    // Program code  
    return 0;  
}
```

In this form, the program runs without any command-line arguments.

2. Command-Line Arguments Form:

C++

```
int main(int argc, char *argv[]) {  
    // Program code  
    return 0;  
}
```

- `int argc`: Stands for "argument count" and represents the number of command-line arguments.
- `char *argv[]`: Stands for "argument vector" and is an array of strings where each element represents a command-line argument.

Command Line Arguments:

- `argv[0]`: The name of the program itself.
- `argv[1]` to `argv[argc-1]`: The actual command-line arguments provided when running the program.

C++

```

int main(int argc, char *argv[]) {
    cout<<"Program Name: "<<argv[0]<<endl;

    if (argc > 1) {
        cout<<"Command-Line Arguments:"<<endl;
        for (int i = 1; i < argc; ++i) {
            cout<<"Argument "<<i<<": "<<argv[i]<<endl;
        }
    } else {
        cout<<"No command-line arguments provided."<<endl;
    }

    return 0;
}

```

```
./program_name arg1 arg2 arg3
```

Output

```

Program Name: ./program_name
Command-Line Arguments:
Argument 1: arg1
Argument 2: arg2
Argument 3: arg3

```

C++

```

int main(int argc , char * argv[]){

    cout<<"Enter no. of agruments";
    cin>>argc;

    for (int i = 0; i < argc; i++)
        cout<<"argv["<<i<<"] : "<<argv[i]<<endl;

    return 0;
}

```

```
./program_name arg1 arg2 arg3 arg4
```

Output

```

Enter no. of agruments : 4
argv[0] : ./program_name

```

```
argv[1] : arg1  
argv[2] : arg2  
argv[3] : arg3
```

When executing this program from the terminal as `./program_name arg1 arg2 arg3 arg4`, the program receives inputs in the form of command-line arguments. Here's the breakdown:

- `./program_name` is `argv[0]`, representing the program's name.
- `arg1`, `arg2`, `arg3`, and `arg4` become `argv[1]`, `argv[2]`, `argv[3]`, and `argv[4]` respectively.

In the provided example, the user is prompted to enter the number of arguments. If the user inputs 4, the program displays `argv[0]` to `argv[3]`. The discrepancy arises because the program name is implicitly assigned to `argv[0]`, leaving the user-entered arguments to start from `argv[1]`.

To align the entered argument count with the displayed indices, users should input 5:

```
./program_name arg1 arg2 arg3 arg4
```

Now, with `argc` set to 5, the program accurately displays `argv[0]` to `argv[4]` based on the user's input.

Scope Resolution Operator

not in syllabus

The scope resolution operator (SRO) in C++ is represented by two colons (::) and is used to specify the scope of a variable or a function. Here's what it does:

1. **Accessing Members of a Namespace:** When used at the global level or within a namespace, the SRO can access members (variables, functions, classes) defined within that namespace.

C++

```
namespace MyNamespace {
    int x;
    void foo();
}

int main() {
    MyNamespace::x = 10;    // Accessing variable x in MyNamespace
    MyNamespace::foo();     // Calling function foo() in MyNamespace
    return 0;
}
```

2. **Accessing Static Members of a Class:** When used with classes, SRO accesses static members (variables or functions) of the class.

C++

```
class MyClass {
public:
    static int count;
    static void displayCount();
};

int MyClass::count = 0;

void MyClass::displayCount() {
    cout << "Count: " << count << endl;
}

int main() {
    MyClass::count = 5;        // Accessing static variable count
    MyClass::displayCount();   // Calling static function displayCount()
}
```

```
return 0;
}
```

3. **Accessing Nested Classes:** SRO can also be used to access nested classes within a class.

C++

```
class Outer {
public:
    class Inner {
public:
        void display() {
            cout << "Inner class" << endl;
        }
    };
};

int main() {
    Outer::Inner obj;    // Accessing nested class Inner
    obj.display();       // Calling display function of Inner class
    return 0;
}
```

4. **Namespace Localisation:** SRO is also used for explicit specification of namespaces to ensure unambiguous identifier resolution and to avoid conflicts.

C++

```
#include <iostream>

int main(){

    std::cout<<"Hello World !"<<std::endl;

    return 0;
}
```

In summary, the scope resolution operator is a powerful tool in C++ for accessing members in namespaces, classes, and nested scopes, allowing for better organisation and management of code.

File Handling

File handling in C++ involves the management and manipulation of files to read or write data. C++ provides the `<fstream>` library, which includes classes like `ifstream`, `ofstream`, and `fstream` for handling file operations.

Key Concepts and Components:

1. File Streams:

- C++ provides three main classes for file handling:
 - `ifstream` (input file stream): Used for reading data from a file.
 - `ofstream` (output file stream): Used for writing data to a file.
 - `fstream` (file stream): Combines functionalities of both `ifstream` and `ofstream` and can be used for both reading and writing.

2. Opening and Closing Files:

- Before performing file operations, a file must be opened using the `open` method. The method requires the file name and, optionally, additional parameters like mode (`ios::in`, `ios::out`, `ios::app`, etc.).
- After completing file operations, it's essential to close the file using the `close` method.

C++

```
ofstream outputFile; // Object and Object Name
outputFile.open("example.txt");
// Perform file operations
outputFile.close();
```

3. Reading from Files:

- Use the `>>` or `getline` operators to read data from an input file stream (`ifstream` or `fstream`).

C++

```
ifstream inputFile; // Object and Object Name
inputFile.open("example.txt");
int number;
inputFile >> number;
// or
string line;
getline(inputFile, line);
```

4. Writing to Files:

- Use the << operator to write data to an output file stream (`ofstream` or `fstream`).

C++

```
ofstream outputFile;  
outputFile.open("example.txt");  
int number = 42;  
outputFile << number;
```

5. File Modes:

- File modes (e.g., `ios::in`, `ios::out`, `ios::app`) specify the intended use of the file. Choose the appropriate mode based on whether you want to read, write, or append to a file.

C++

```
ofstream outputFile;  
outputFile.open("example.txt", ios::out | ios::app);
```

Example Codes :

C++

```
// #include <fstream>  
  
fstream fw;  
  
fw.open("myfile.txt", ios::out);  
  
fw<<"text inside file";  
fw.close();
```

C++

```
// #include <fstream>  
  
fstream fw;  
  
int a = 10;  
int b = 20;  
fw.open("myfile.txt", ios::out);  
  
fw<<a<<b;
```

```
fw.close();
```

C++

```
// #include <fstream>

fstream fw;
char ch;
fw.open("myfile.txt", ios::in);

fw>>ch;
cout<<ch;

fw.close();
```

C++

```
// #include <fstream>

fstream fw;
char ch;
fw.open("myfile.txt", ios::in);

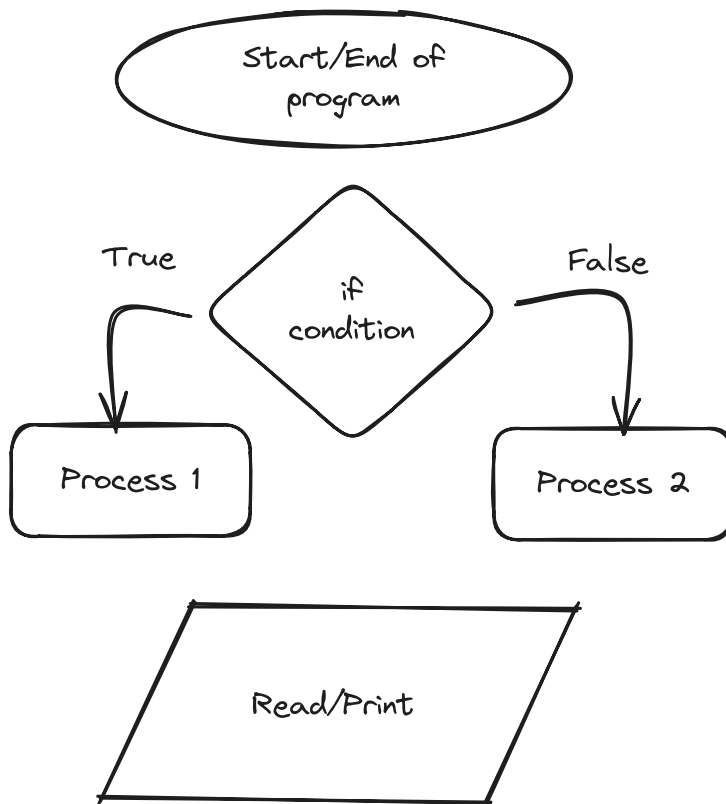
while (1){
    fw>>ch;
    if(fw.eof())
        break;

    else
        cout<<ch; // PRINTING THE FILE CHARACTER BY CHARACTER
}

fw.close();
```

Flow Chart

Pictorial representation of flow of sequence of execution of program.



C++

```
int n;  
cin>>n;  
if (n%2 == 0)  
    cout<<"Even";  
else  
    cout<<"Odd";
```

C++

```
int main(){  
    int a, b, c;  
    cin>>a;  
    cin>>b;  
    c = a+b;  
    cout<<c;  
    return 0;  
}
```

Exception Handling

C++

```
int a,b;
cout<<"Enter a : ";
cin>>a;
cout<<"Enter b : ";
cin>>b;
cout<<"a/b : "<<a/b<<endl;
cout<<"Sucessfully Terminated"<<endl;
```

Output

```
Enter a : 81
Enter b : 3
a/b : 27
Sucessfully Terminated
```

C++

```
int a,b;
cout<<"Enter a : ";
cin>>a;
cout<<"Enter b : ";
cin>>b;
cout<<"a/b : "<<a/b<<endl;
cout<<"Sucessfully Terminated"<<endl;
```

Output

```
Enter a : 15
Enter b : 0
```

13 `cout<<"a/b : "<<a/b<<endl;`

Exception has occurred. ×
Arithmetic exception

syntax

C++

```
try{
code in which exception may occur
throw() keyword
}
catch{
handling of exception
}
```

C++

```
int a,b;
cout<<"Enter a : ";
cin>>a;
cout<<"Enter b : ";
cin>>b;
try{
if (b != 0){
cout<<"a/b : "<<a/b<<endl;
}
else{
throw(b);
}
}
catch(int i){
cout<<"Cannot Divide by Zero"<<endl;
}
cout<<"Sucessfully Terminated"<<endl;
```

Output

```
Enter a : 15
Enter b : 0
Cannot Divide by Zero
Sucessfully Terminated
```

Furthermore, you can define your own errors and exceptions by using `#include <exception>` header file and inheriting the `std::exception` class into your custom exception.

Storage Classes

Storage classes are used to define the characteristics of a variable or function, determining aspects such as its lifetime, visibility, default value, and storage location. There are several types of storage classes, each serving different purposes:

Types of Storage Classes:

1. **Auto:**

- Automatic variables have local scope.
- They have a default garbage value.

2. **Extern:**

- The initial value of an extern variable is zero.
- Extern variables have global scope.

3. **Static:**

- Static variables are declared and initialised only once, regardless of how many times the function is called. They act as class variables.
- The initial value of a static variable is zero.
- The lifetime of a static variable is throughout the program.
- Visibility of a static variable is local to the function where it is declared.

4. **Mutable:**

- Mutable is used with a constant variable.
- It allows the value of the constant variable to be modified, even though constant variables cannot typically be changed.

5. **Register:**

- Register storage class variables are stored in the CPU register instead of RAM for faster access.
- If the register is not available, the variable may be stored in RAM.
- The value of a register variable cannot be accessed using pointers.

Automatic

C++

```
#include <iostream>
using namespace std;

int main(){
    int x;
    cout<<"x (in main fn, declared in main fn) : "<<x<<endl;  // GARBAGE VALUE
    return 0;
}
```

Output

```
x (in main fn, declared in main fn) : 264151
```

C++

```
#include <iostream>
using namespace std;

int main(){
    void show();

    int x;
    cout<<"x (in main fn, declared in main fn) : "<<x<<endl;  // GARBAGE VALUE

    show();
    return 0;
}

void show(){
    // cout<<"x (in show fn, declared in main fn) : "<<x<<endl;  CANNOT DO
    BEACUSE `x` HAS LOCAL SCOPE
}
```

Output

```
x (in main fn, declared in main fn) : 626491
```

Extern

C++

```
#include <iostream>
using namespace std;

int n = 10;
int main(){

void show();

cout<<"n (in main fn, declared and initialised globally) : "<<n<<endl;
show();

return 0;
}

void show(){
cout<<"n (in show fn, declared and initialised globally) : "<<n<<endl;
}
```

Output

```
n (in main fn, declared and initialised globally) : 10
n (in show fn, declared and initialised globally) : 10
```

C++

```
#include <iostream>
using namespace std;

int n = 20;
int main(){

void show();

int n;
cout<<"n (in main fn, declared globally, redeclared in main fn) : "
<<n<<endl;

show();
return 0;
}

void show(){
```

```
cout<<"n (in show fn, declared globally, redeclared in main fn) : "  
<<n<<endl;  
}
```

Output

```
n (in main fn, declared globally, redeclared in main fn) : 64829  
n (in show fn, declared globally, redeclared in main fn) : 20
```

C++

```
#include <iostream>  
using namespace std;  
  
int n = 20;  
int main(){  
  
void show();  
  
int n = 10;  
cout<<"n (in main fn, declared globally, redeclared and initialised in  
main fn) : "<<n<<endl;  
  
show();  
return 0;  
}  
  
void show(){  
cout<<"n (in show fn, declared globally, redeclared and initialised in  
main fn) : "<<n<<endl;  
}
```

Output

```
n (in main fn, declared globally, redeclared and initialised in main fn) :  
10  
n (in show fn, declared globally, redeclared and initialised in main fn) :  
20
```

C++

```
#include <iostream>  
using namespace std;  
  
int n = 20;  
int main(){
```

```
void show();

extern int n;
cout<<"n (in main fn, declared globally, extern redeclared in main fn) : "
<<n<<endl;

show();
return 0;
}

void show(){
cout<<"n (in show fn, declared globally, extern redeclared in main fn) : "
<<n<<endl;
}
```

Output

```
n (in main fn, declared globally, extern redeclared in main fn) : 20
n (in show fn, declared globally, extern redeclared in main fn) : 20
```

Static

C++

```
#include <iostream>
using namespace std;

int main(){
    void show();

    show ();
    show ();
    return 0;
}

void show(){
    int n = 0;
    n++;
    cout<<"n (in show fn, declared and initialised as 0 in show fn) : "
    <<n<<endl;
}
```

Output

```
n (in show fn, declared and initialised as 0 in show fn) : 1
n (in show fn, declared and initialised as 0 in show fn) : 1
```

C++

```
#include <iostream>
using namespace std;

int main(){
    void show();

    show ();
    show ();
    show ();
    return 0;
}

void show(){
    static int n = 0;
    n++;
    cout<<"n (in show fn, static declared and initialised as 0 in show fn) : "
```

```
<<n<<endl;  
}
```

Output

```
n (in show fn, static declared and initialised as 0 in show fn) : 1  
n (in show fn, static declared and initialised as 0 in show fn) : 2  
n (in show fn, static declared and initialised as 0 in show fn) : 3
```

C++

```
#include <iostream>  
using namespace std;  
  
int main(){  
void show();  
  
show ();  
show ();  
show ();  
return 0;  
}  
  
void show(){  
static int n;  
n++;  
cout<<"n (in show fn, static declared in show fn) : "<<n<<endl;  
}
```

Output

```
n (in show fn, static declared in show fn) : 1  
n (in show fn, static declared in show fn) : 2  
n (in show fn, static declared in show fn) : 3
```

Register

C++

```
#include <iostream>
using namespace std;

int main(){

    register int n;
    cout<<"n (in main fn, register declared in main fn) : "<<n<<endl;
    cout<<"&n (in main fn, register declared in main fn) : "<<&n<<endl;

    return 0;
}
```

Output

```
n (in main fn, register declared in main fn) : 0
&n (in main fn, register declared in main fn) : 0x7fffffff6b4
```

In ISO C++17 and later standards, the use of `register` has been deprecated or disallowed because compilers typically make their own decisions regarding register allocation based on optimisation strategies.

Mutable

C++

```
#include <iostream>
using namespace std;

struct book{
int x = 10;
}; // REGULAR GLOBAL STRUCTURE DEFINITION

int main(){

struct book b; // REGULAR DECLARATION OF GLOBAL STRUCTURE `book` WITH
VARIABLE `b` IN `main()`
cout<<"b.x (global structure, auto member, regular structure variable) : "
<<b.x<<endl;
b.x = 90; // MODIFYING THE VALUE INSIDE STRUCTURE VARIABLE `b`
cout<<"modified b.x (global structure, auto member, regular structure
variable) : "<<b.x<<endl;

return 0;
}
```

Output

```
b.x (global structure, auto member, regular structure variable) : 10
modified b.x (global structure, auto member, regular structure variable) :
90
```

C++

```
#include <iostream>
using namespace std;

struct book{
int x = 10;
}; // REGULAR GLOBAL STRUCTURE DEFINITION

int main(){

const struct book b; // CONSTANT DECLARATION OF GLOBAL STRUCTURE `book`
WITH VARIABLE `b` IN `main()`
cout<<"b.x (global structure, auto member, constant structure variable) :
"<<b.x<<endl;
```

```
// b.x = 90;  CANNOT MODIFY THE VALUE INSIDE STRUCTURE VARIABLE `b`

return 0;
}
```

Output

```
b.x (global structure, auto member, constant structure variable) : 10
```

C++

```
#include <iostream>
using namespace std;

struct book{
mutable int x = 10;
}; // GLOBAL STRUCTURE DEFINITION WITH `mutable` VARIABLE

int main(){

const struct book b; // CONSTANT DECLARATION OF GLOBAL STRUCTURE `book`
WITH VARIABLE `b` IN `main()`
cout<<"b.x (global structure, mutable member, constant structure variable)
: "<<b.x<<endl;
b.x = 90; // MODIFYING THE VALUE INSIDE STRUCTURE VARIABLE `b`, CAN BE
DONE BECAUSE MUTABLE MEMBER CAN BE MODIFIED IN CONSTANT DECLARED STRUCTURE
cout<<"modified b.x (global structure, mutable member, constant structure
variable) : "<<b.x<<endl;

return 0;
}
```

Output

```
b.x (global structure, mutable member, constant structure variable) : 10
modified b.x (global structure, mutable member, constant structure
variable) : 90
```